

C102「AI・機械学習：ニューラルネットワークによる教師あり  
学習」

学生番号：242C2016 氏名：奥村直

知的システム工学科システム制御コース

2025年12月15日

# 1 目的

本演習の目的は、実際に Matlab のプログラミングを行うことにより、AI・機械学習の基礎の一つである「ニューラルネットワークによる教師あり学習」を理解することである。

## 2 第1週：最急降下法

### 2.1 演習課題 1.1

#### 2.1.1 演習課題 (a)

(a) 初期値  $w$  やステップ幅  $ep$  を様々な値に変更し、最急降下法での収束がどのように変わるか確かめよ。初期値は描画範囲（-5～5 の範囲）で 2 刻みくらいで変更する。ステップ幅  $ep$  は 0.8, 0.5, 0.2, 0.1, 0.05, 0.01 などに変更する。必要に応じて、反復回数  $iters$  も変更する。

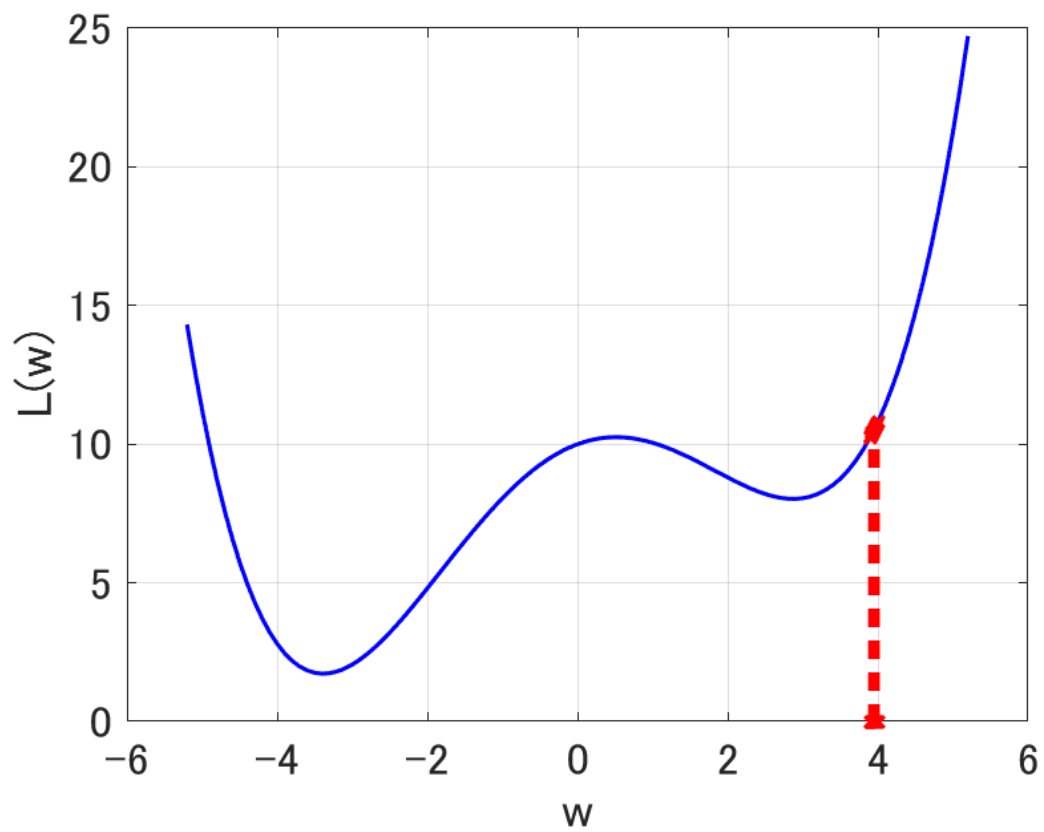


図 2.1:  $w=4, ep=0.001, iters=20$  のときのグラフ

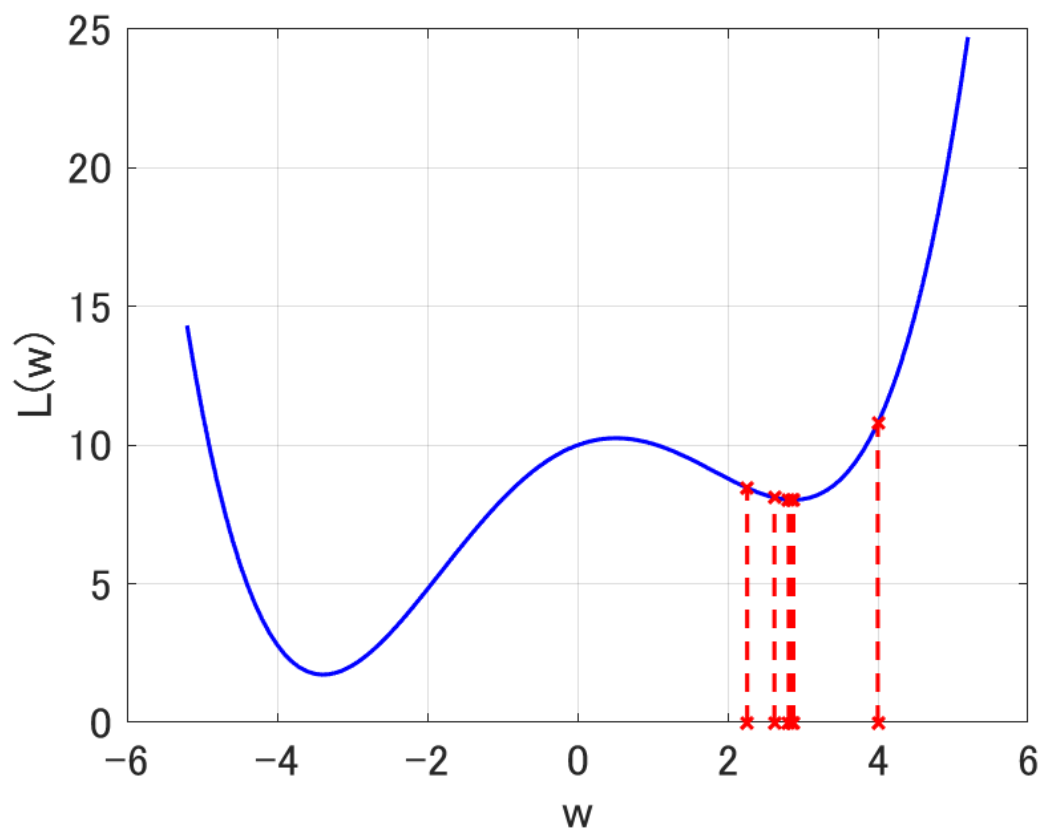


図 2.2:  $w=4, ep=0.3, iters=20$  のときのグラフ

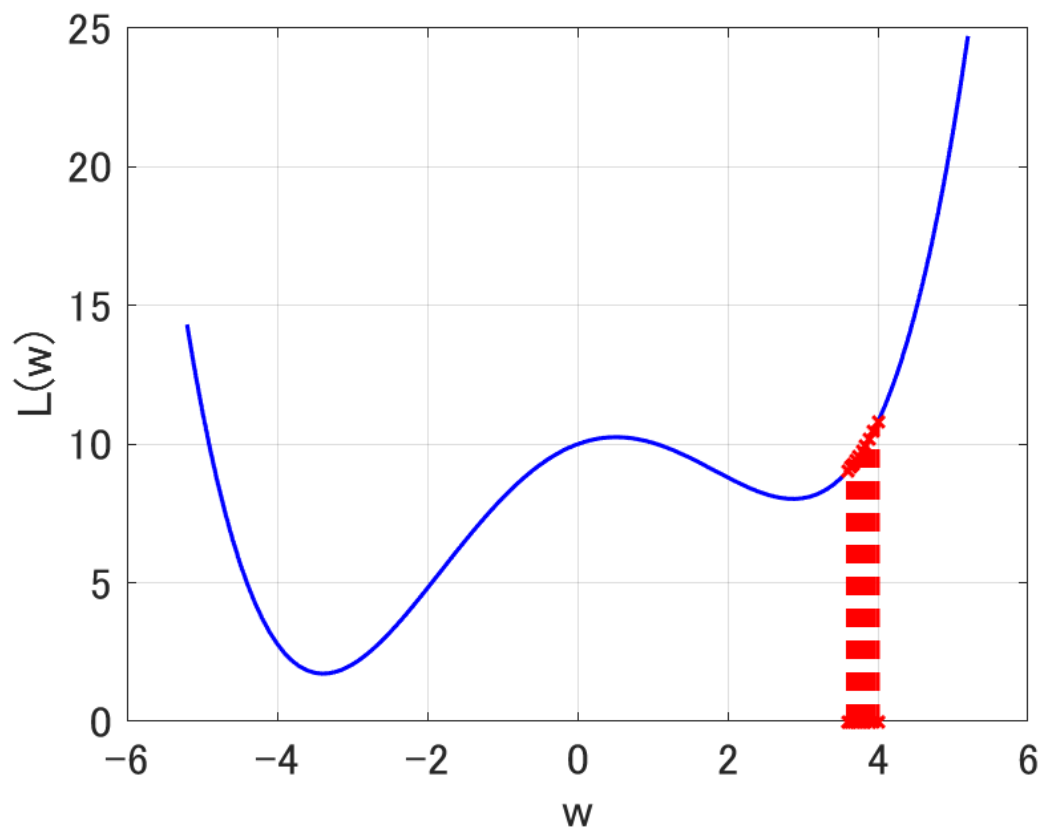


図 2.3:  $w=4, ep=0.01, iters=10$  のときのグラフ

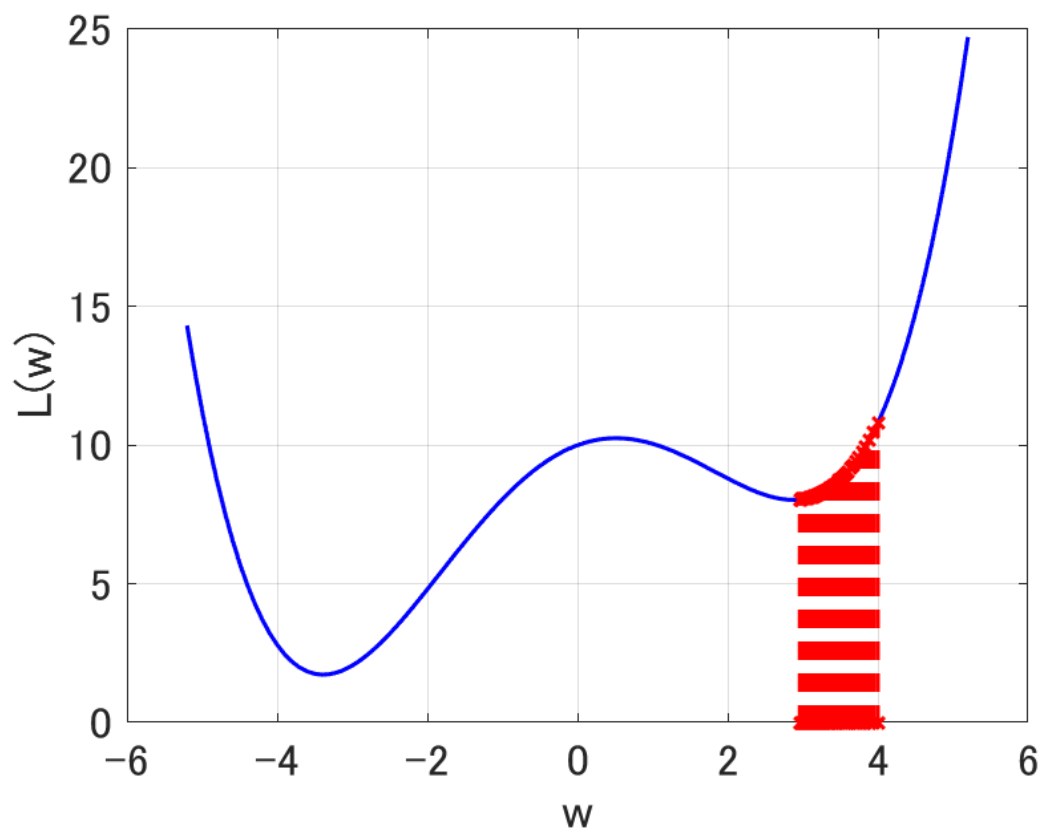


図 2.4:  $w=4, ep=0.01, iters=70$  のときのグラフ

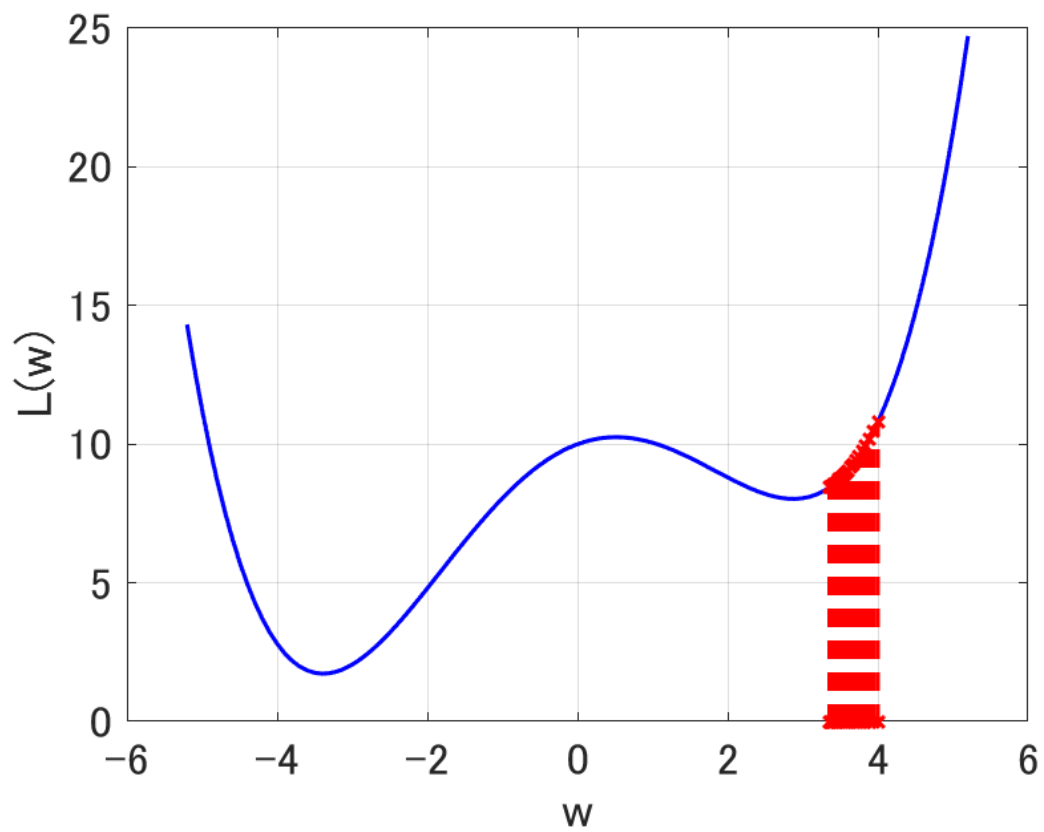


図 2.5:  $w=4, ep=0.01, iters=20$  のときのグラフ

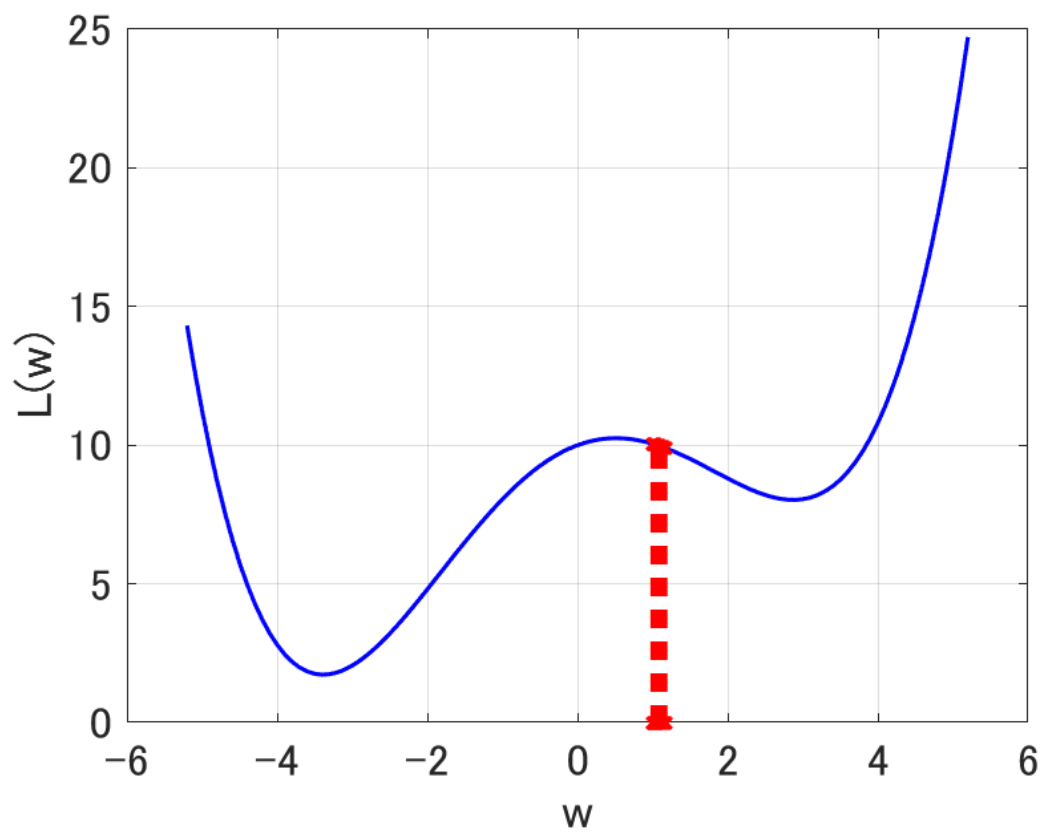


図 2.6:  $w=1, ep=0.01, iters=20$  のときのグラフ

### 2.1.2 演習課題 1.1 (b)

(b) 目的関数が

$$L(w) = \sin(2w) + \frac{w^2}{8} + 2 \quad (2.1)$$

の場合について、プログラムを変更して「saikyu12.m」とし、(a)と同じように様々な初期値とステップ幅に対してどのような振る舞いになるかを確認せよ。この場合の勾配  $\frac{\partial}{\partial w} L(w)$  は自分で計算すること。  
目的関数  $L(w)$  の勾配を求めると、

$$\frac{\partial}{\partial w} L(w) = 2 \cos 2w + \frac{w}{4} \quad (2.2)$$

であることが分かるから、saikyu11.m の

```
function g = grad(w)
g = w.^3/5 - 2*w + 1;
end
```

の関数を以下のように変更し、

```
function g = grad(w)
g = 2 * cos(2 * w) + w/4;
end
```

saikyu12.m の全体のソースコードを次に示す。

Listing 2.1: saikyu12

```
1  %%%% 以下を入力 %%%%
2  % プログラム1.1: saikyu11.m
3  %
4  % 変数の目的関数対する最急降下法1
5  %
6  % すべての変数の値を初期化する。グラフをすべて閉じる。
7  clearvars, close all
8  % ← (A)
9  w = 4;
10 ep = 0.1;
11 iters = 20;
12 ws = w;
13 for k = 1:iters
14     g = grad(w);
15     w = w - ep*g;
16     ws = [ws; w];
17 end
18 %
19 %% 目的関数のグラフを描く
20 % w: 変数. 範囲: から刻みでまでの範囲 -5.20.15.2
21 % Lw: 目的関数の値
22 w = -5.2:0.1:5.2;
23 Lw = obj_func(w);
24 plot(w, Lw, 'b-', 'LineWidth', 1.5), grid on, hold on
25 xlabel('w'), ylabel('L(w)')
26 set(gca, 'FontSize', 16) % フォントサイズをポイントにする. 16
27 % ← (B)
28 for k = 1:iters
29     yk = obj_func(ws(k));
30     plot([ws(k), ws(k)], [0, yk], 'rx--', 'LineWidth', 1.5);
31 end
32 %
```

```

33 %% 目的関数 (objective) の計算function
34 function y = obj_func(w)
35 y = sin(2 * w) + w.^2/8 + 2;
36 end
37 % ← (C)
38 function g = grad(w)
39 g = 2 * cos(2 * w) + w/4;
40 end
41 %%%% 以上を入力 %%%%

```

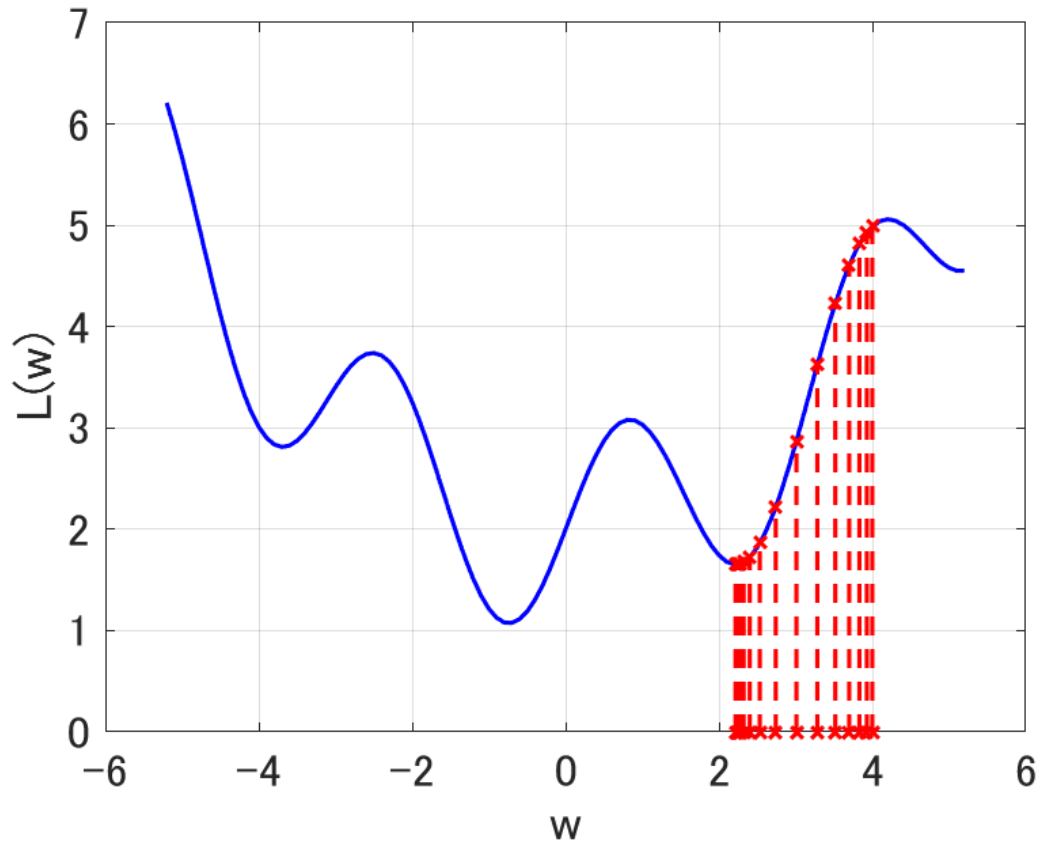


図 2.7: saikyu12

### 2.1.3 考察

(a) この演習課題をやるにあたり、 $w$ 、 $ep$ 、 $iters$  の値をそれぞれひとつだけ変化させ、出力されるグラフにどんな影響があるかを見た。 $ep$  を変化させたとき、 $ep$  が大きければ降下が早く、小さければ降下が遅いことが分かった (図 2.1、図 2.2)。次に  $w$  を変化させると、ただ初めに調べる場所が変化した (図 2.5、図 2.6) 期値よりも大きすぎると、極値を通り過ぎて反対に戻っている様子を伺うことができる。また、反復回数  $iters$  を変化させると探索する回数が変わることが分かる。(図 2.3、図 2.4)

(b) 目的関数を変更したためもちろん描画されるグラフも大きく異なっている。また、三角関数が含まれていることで波形がやや複雑になっている。描画したグラフを図 2.7 に示す。

## 2.2 演習課題 1.2

saikyu21.m のプログラムを以下にしめす。

Listing 2.2: saikyu21.m

```

1 % プログラム1.2: saikyu21.m
2 %
3 % 変数の目的関数対する最急降下法2
4 %
5 % すべての変数の値を初期化する。グラフをすべて閉じる。
6 clearvars, close all
7 % ← (A)
8 %% 最急降下法
9 % 初期値, ステップ幅, 反復回数の設定
10 w = [1,1]; % の初期値w
11 ep = 0.1; % ステップ幅
12 iters = 20; % 反復回数 (number of ) iterations
13 ws = w; % ws: w のすべての値を保存する
14 % 更新の反復
15 for i=1:iters
16 gk = grad(w(1),w(2)); % 勾配の計算
17 w = w - ep*gk; % 更新
18 ws = [ws; w]; % 反復点の保存
19 end
20
21 %% 目的関数のグラフを描く
22 % w: 変数. 範囲: から刻みでまでのメッシュ範囲 -60.256
23 % LW: 目的関数
24 [w1, w2] = meshgrid(-6:0.25:5.5);
25 LW = obj_func(w1, w2);
26 % 目的関数の描画
27 sc = meshc(w1, w2, LW); hold on
28 sc(2).LevelList = 2:2:40; % (w1, w2) 平面での等高線の間隔を指定
29 xlabel('w1'), ylabel('w2'), zlabel('L(W)')
30 set(gca, 'FontSize', 14)
31 % ← (B)
32 %% 結果の描画
33 for i=1:iters
34 yk = obj_func(ws(i,1), ws(i,2));
35 plot3([ws(i,1), ws(i,1)], [ws(i,2), ws(i,2)], [0, yk], 'rx--', 'LineWidth', 1.5)
36 end
37 %% (w1, w2) 平面での等高線の描画
38 figure % 新しい描画ウィンドウを開く
39 contour(w1, w2, LW, 25), hold on
40 plot(ws(:,1), ws(:,2), 'rx-', 'LineWidth', 1.5)
41 xlabel('w1'), ylabel('w2')
42 set(gca, 'FontSize', 14)
43 %% 目的関数の計算
44 function y = obj_func(w1, w2)
45 y = (w1.^4)/24 + (w2.^4)/32 - w1.^2 - w2.^2 + w1 + w2 + 25;
46 end
47 %
48 %% 勾配の計算
49 function g = grad(w1, w2)
50 g = [(w1.^3)/6 - 2*w1 + 1, (w2.^3)/8 - 2*w2 + 1];
51 end
52 %

```

(a) 初期値  $w$  やステップ幅  $ep$  を様々な値に変更し、最急降下法での収束がどのように変わるかを確かめよ。必要に応じて、反復回数  $iters$  も変更する。



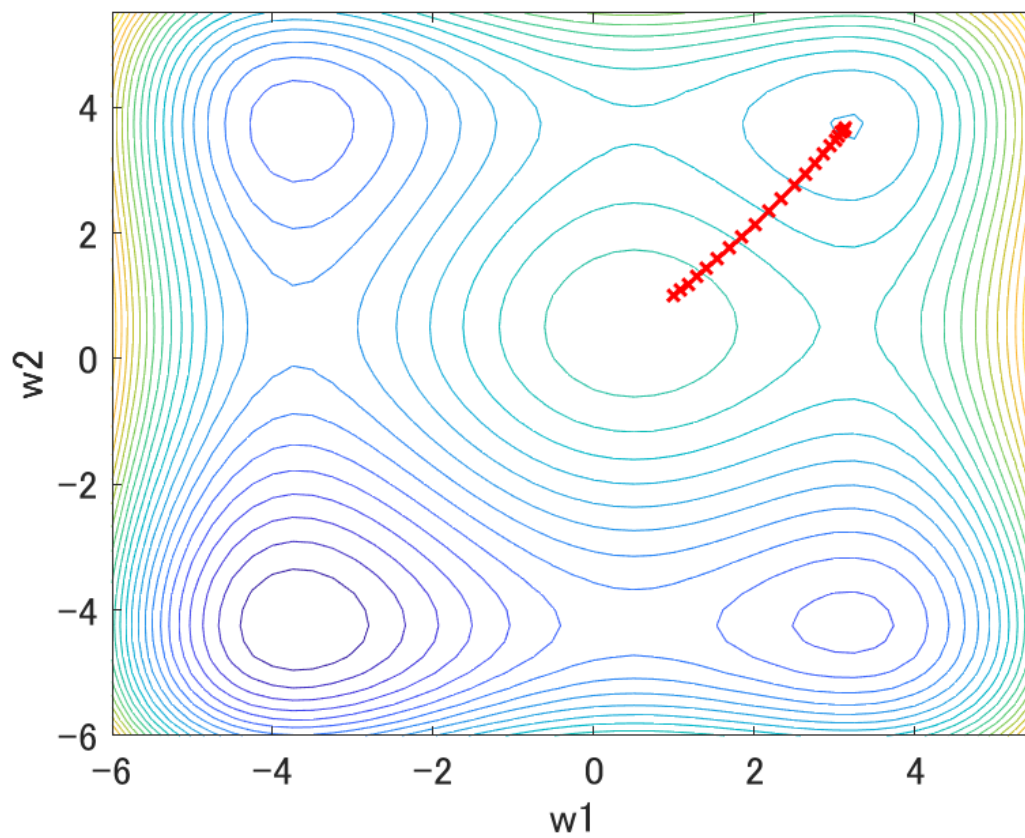


図 2.8: saikyu21

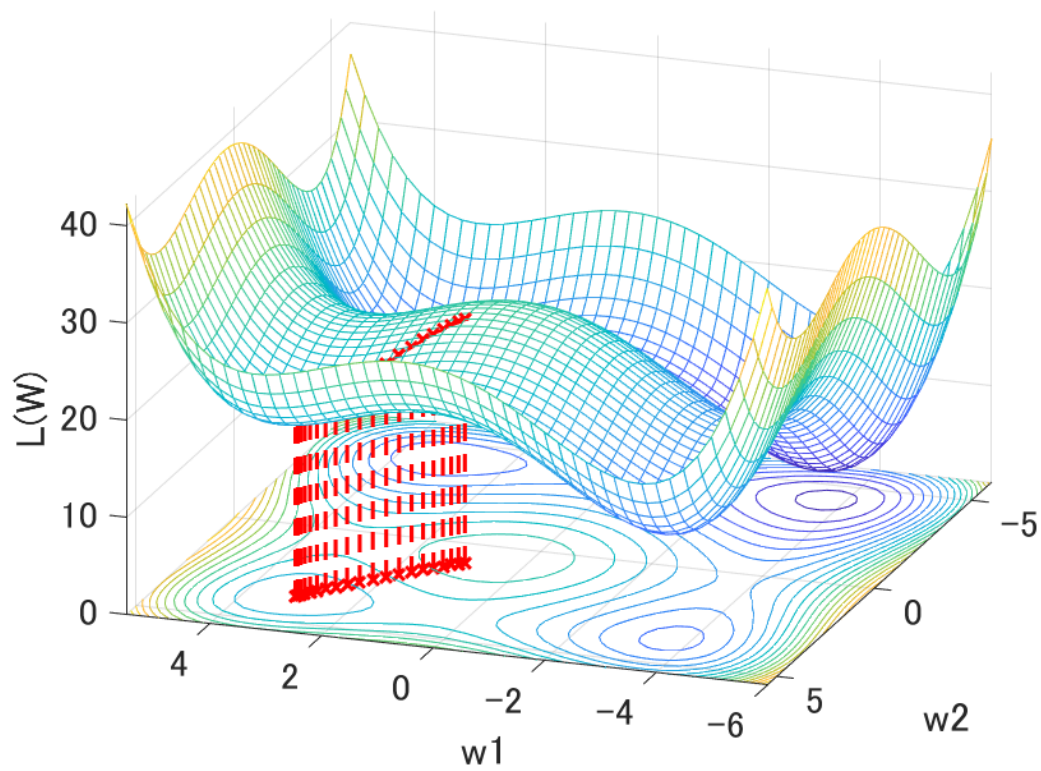


図 2.9: saikyu21

### 2.2.1 考察

完成したプログラムを実行すると画像（図 2.8,2.9）のように出力が得られた。初期値  $W$ 、ステップ幅  $ep$  を様々な値に変更して最急降下法での収束がどのように変わるかを調べたところ、演習課題 1.1 で述べた考察と同じ変化が見られた。

## 3 第2週：パーセプトロン

### 3.1 演習課題 2.1

#### 3.1.1 (a)

(a) 初期値  $W, b$  を変化させて収束の様子を確認する。必要に応じてエポック数を変更する。また、誤差の範囲を変更し、完全にはクラス分類できない場合があることを確かめよ。

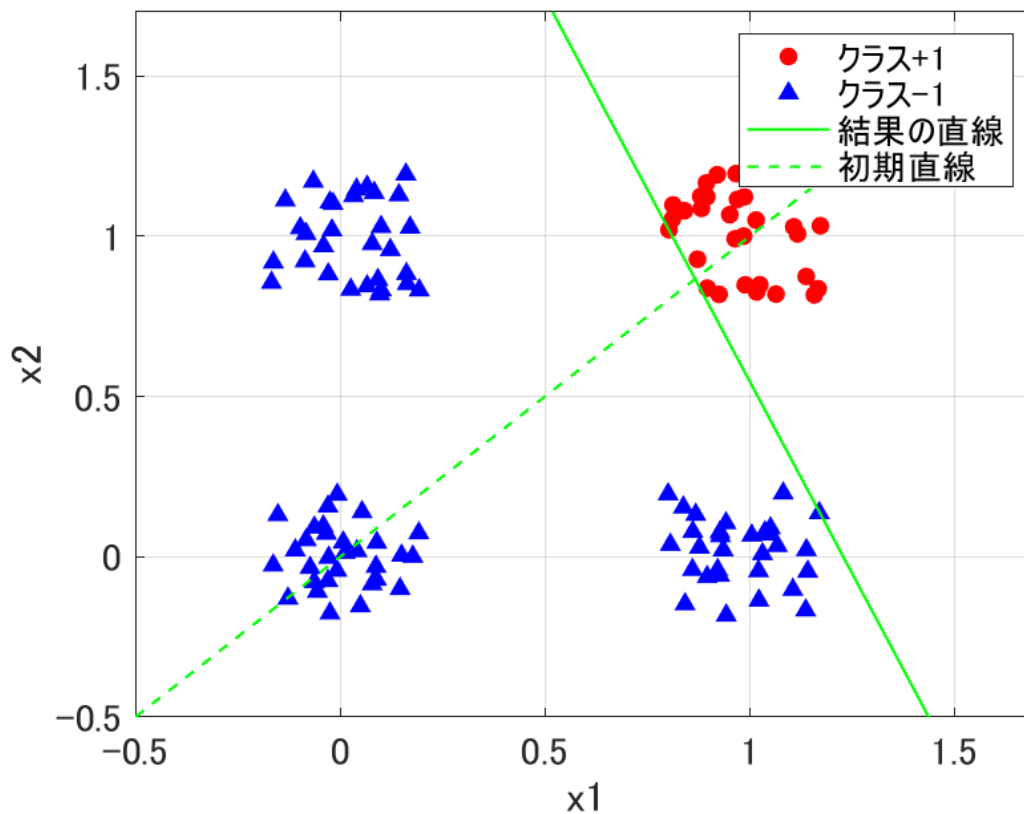


図 3.1:  $er=0.2$

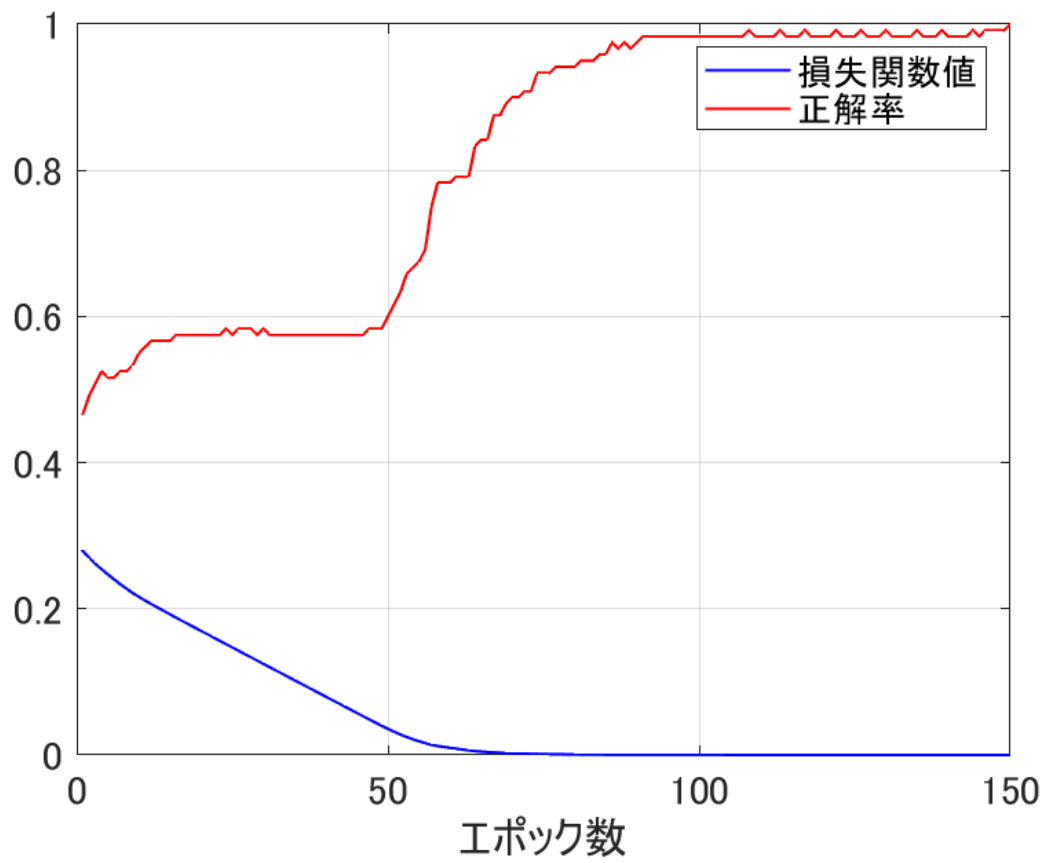


図 3.2:  $er=0.2$

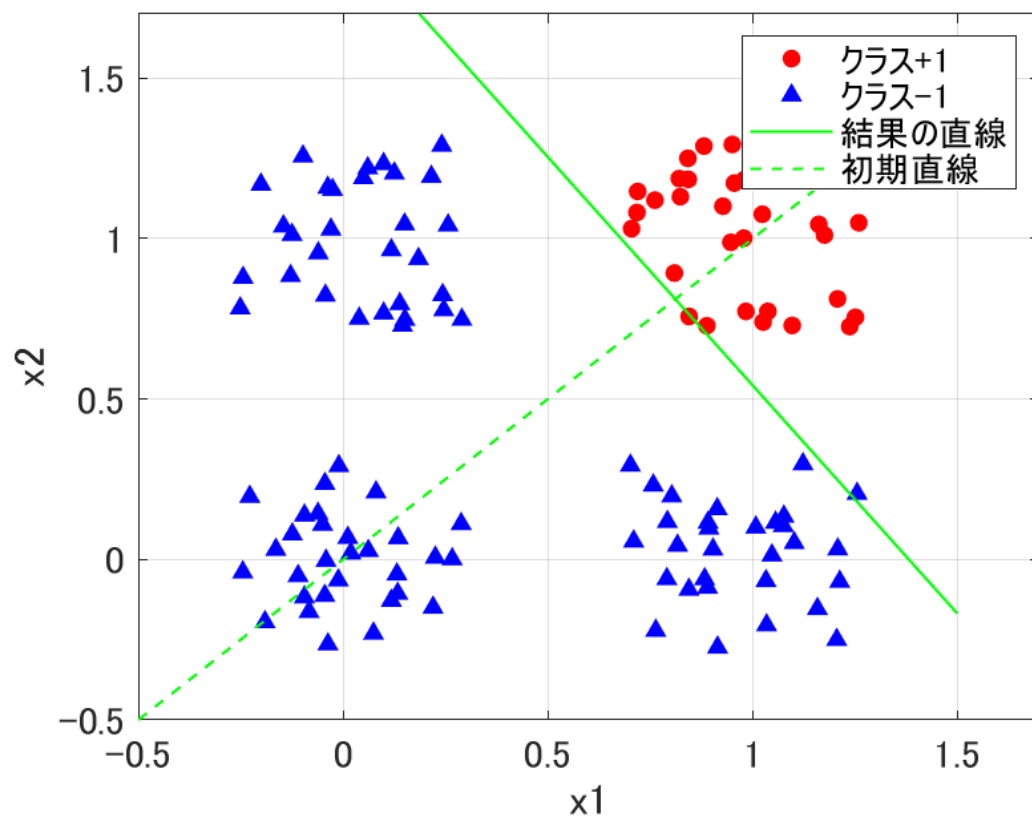


図 3.3:  $er=0.3$

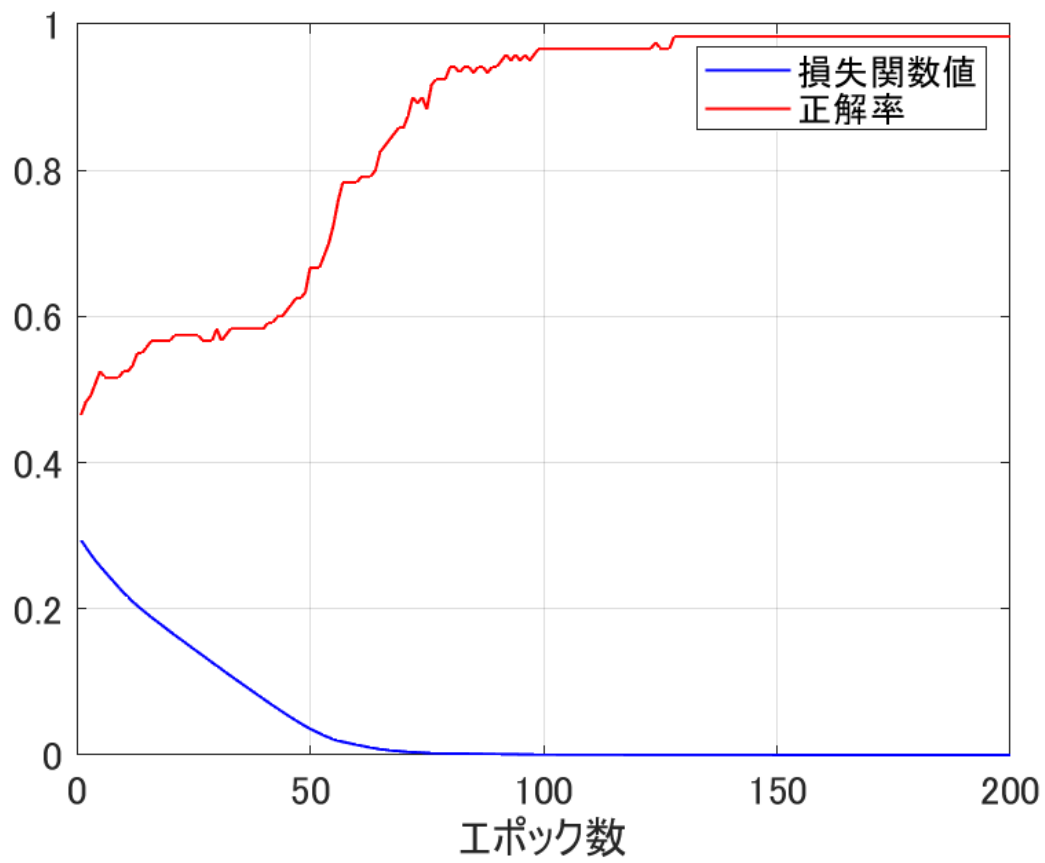


図 3.4:  $er=0.3$

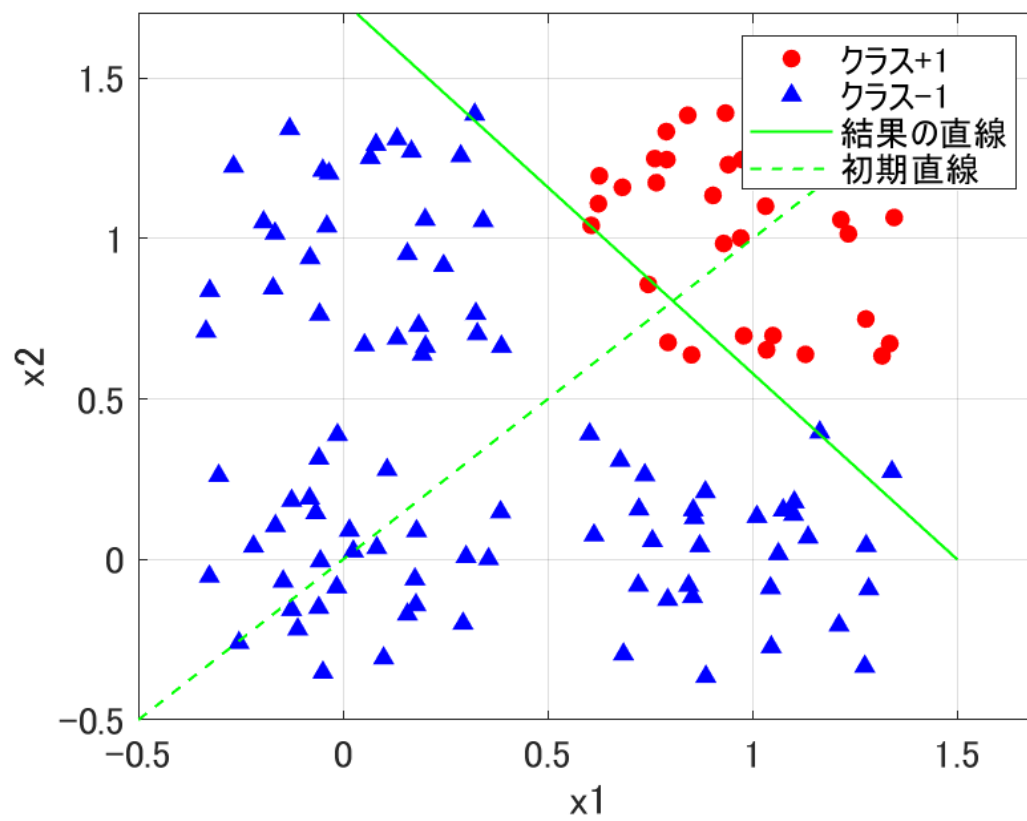


図 3.5:  $er=0.4$

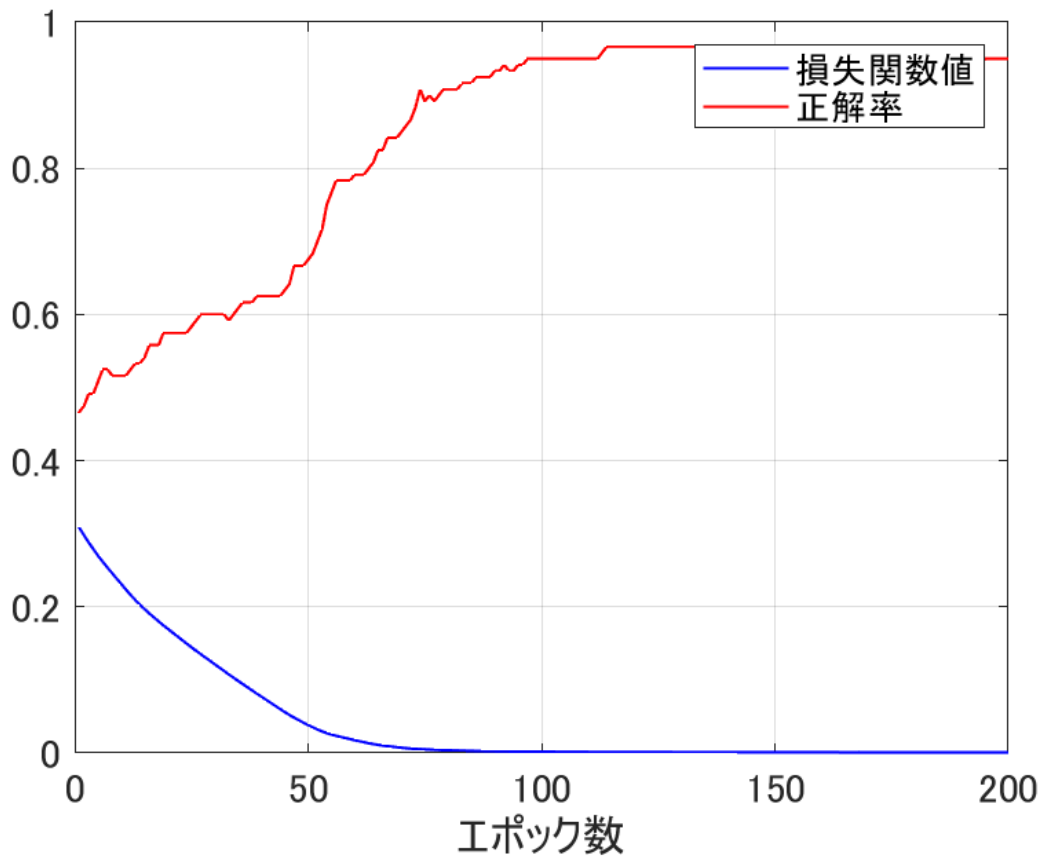


図 3.6:  $er=0.4$

### 3.1.2 考察 (a)

直線の傾きを  $W$  が、切片を  $b$  が表している。また  $er$  は点のばらつきの度合いを表している。 $er$  の値が大きいほどばらつきが大きくなる。私は今回この考察を書くにあたり  $er$  が 0.2, 0.3, 0.4 のときについて調べた。添付している画像の損失関数を見ると、40 から 200 ほどまでには 0 になっていて、期待された値とのずれがないことが見て取れる。このことからこれ以上学習させても正解率が上がることが期待されないため、これ以上のクラスタリングは不可能であると考えられる。

### 3.1.3 (b)

(b) 入力の中ほどにある  $tset$  を  $or$  関係にして、(a) と同様に収束性を確認する。

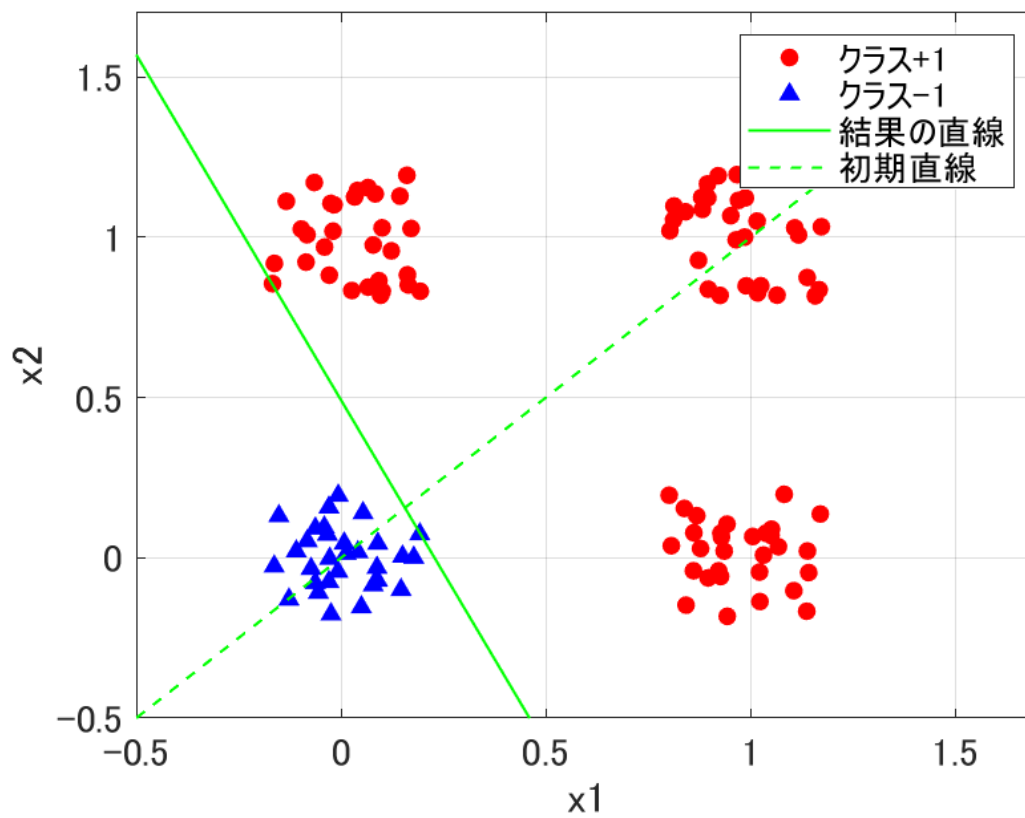


図 3.7: or 関係

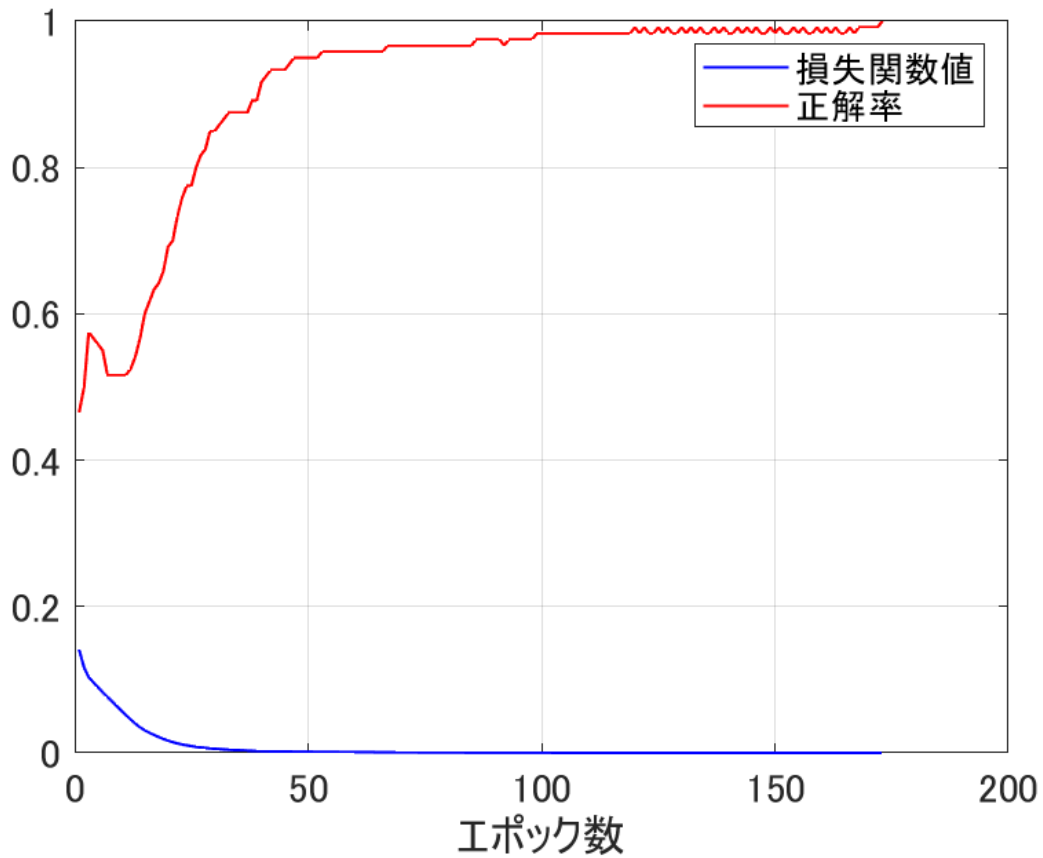


図 3.8: or 関係

### 3.1.4 考察 (b)

and 関係から or 関係に変更して、プログラムを実行した。実行して得られた損失関数と正解率のグラフを見ると、収束の仕方は and 関係のグラフと変わらないように見える。

### 3.1.5 (c)

(c) 上記プログラムをオンライン学習やミニバッチ学習に変更せよ。ミニバッチ学習の場合、例えばバッチサイズを 20 や 30 として実行せよ。

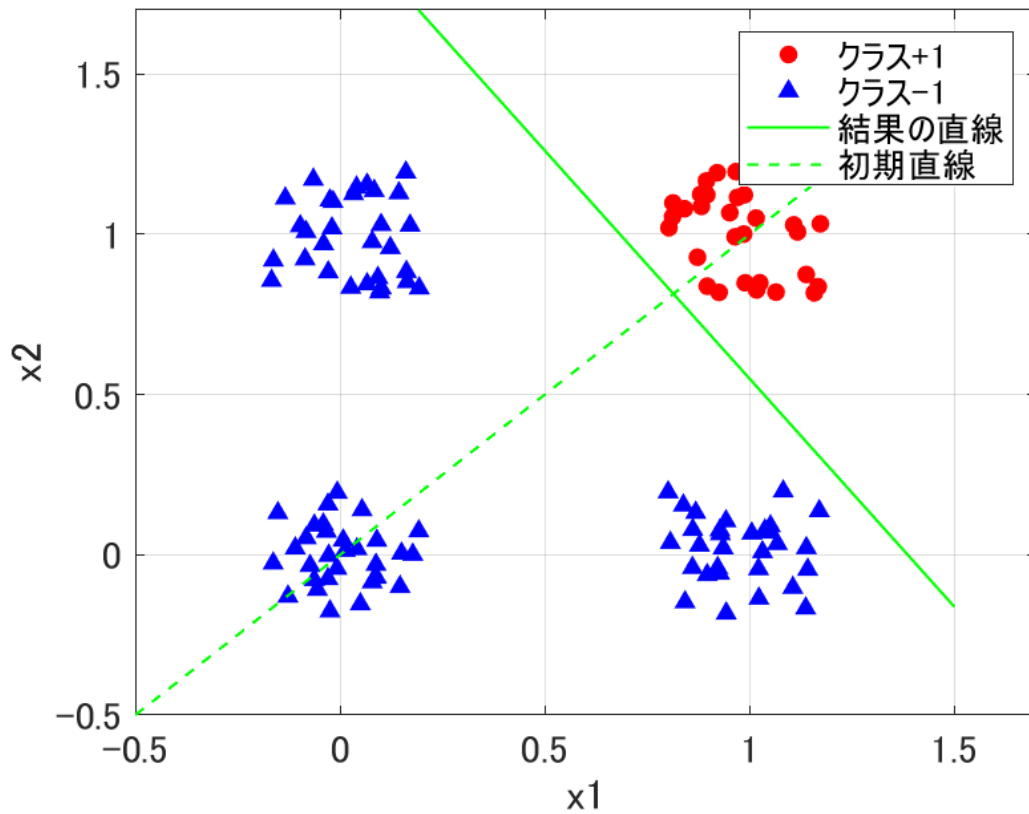


図 3.9: オンライン学習

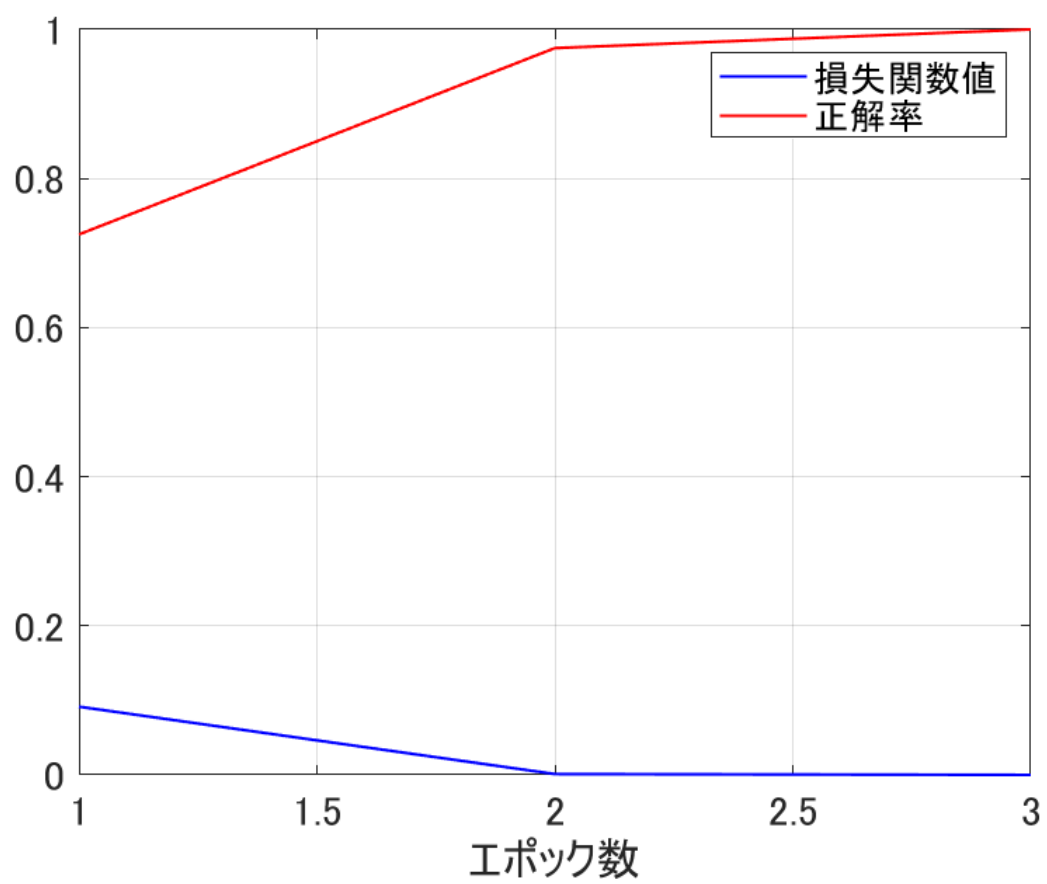


図 3.10: オンライン学習



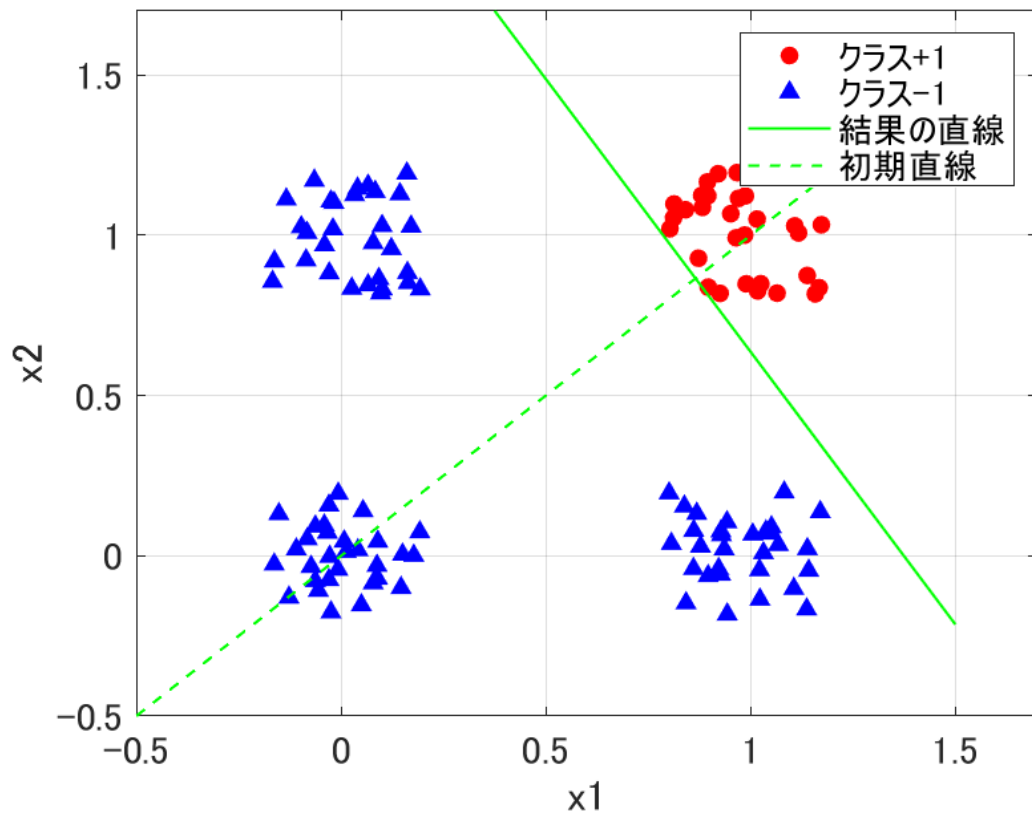


図 3.11: ミニバッチ学習

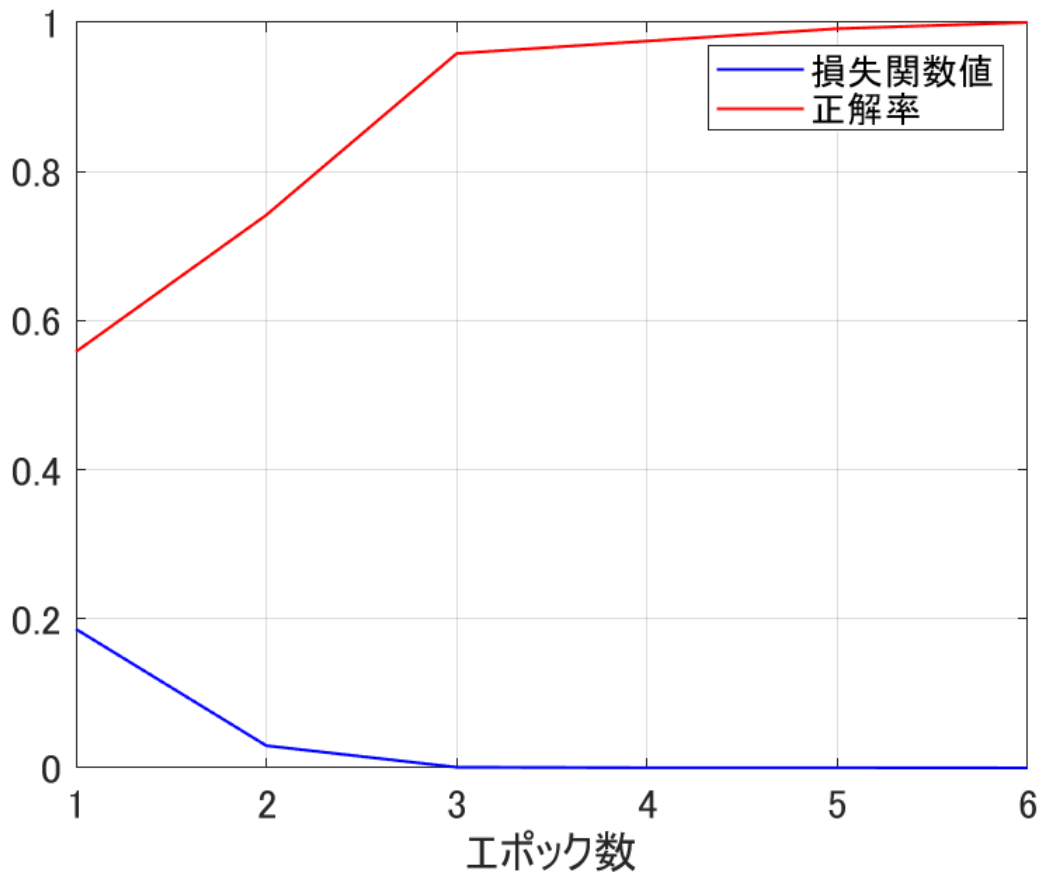


図 3.12: ミニバッチ学習

### 3.1.6 考察 (c)

オンライン学習とミニバッチ学習はどちらもバッチ学習よりもエポック数が少なくなる学習方法ですが、ある一定の区間をまとめてバッチ学習をして、その処理を繰り返しているミニバッチ学習のほうが、オンライン学習よりもエポック数が多くなっている。それぞれのアルゴリズムをみればこの結果が出た理由が良くわかると思った。特にバッチ学習は全データを見ることでエポック数が一番多くなる、という結果が顕著にグラフに表れているよ

うに見える。

## 4 第3週：多層ニューラルネットワーク

### 4.1 演習課題 3.1 (a)

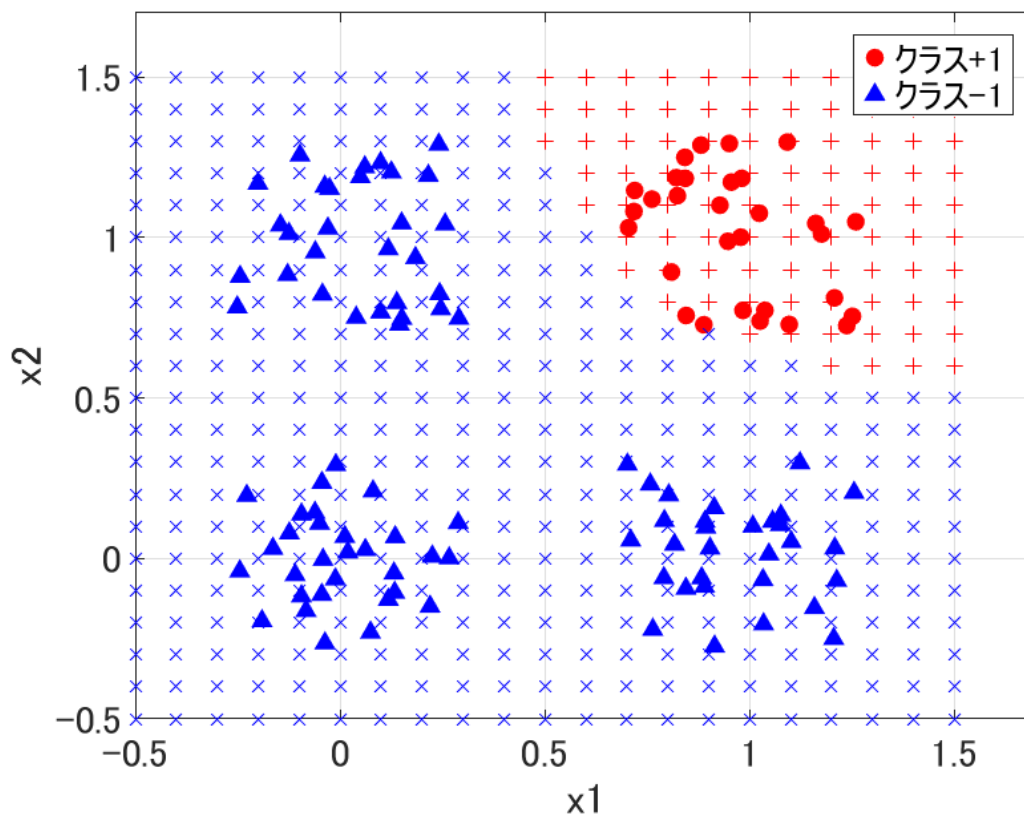


図 4.1: 3 層 NN, dimlayer=50, numEpochs=200, er=0.3

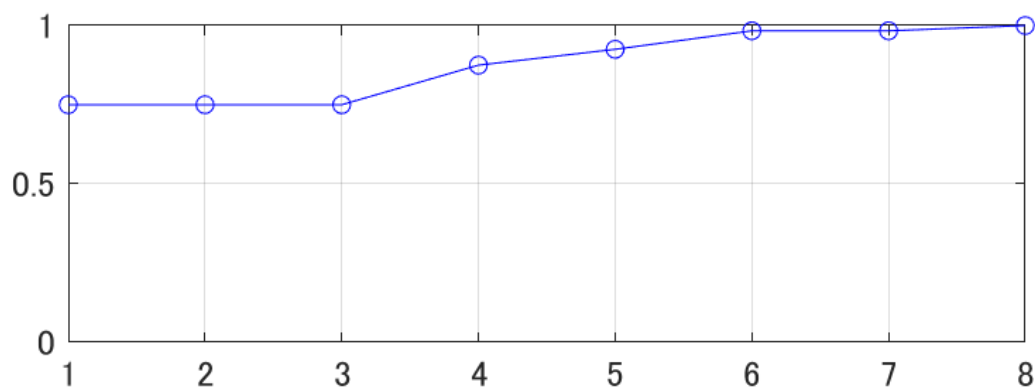


図 4.2: 3 層 NN, dimlayer=50, numEpochs=200, er=0.3

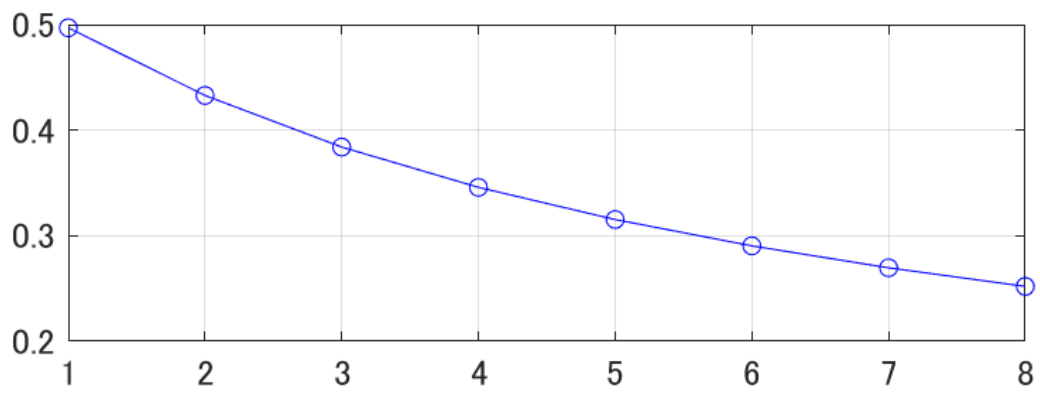


図 4.3: 3 層 NN, dimlayer=50, numEpochs=200, er=0.3

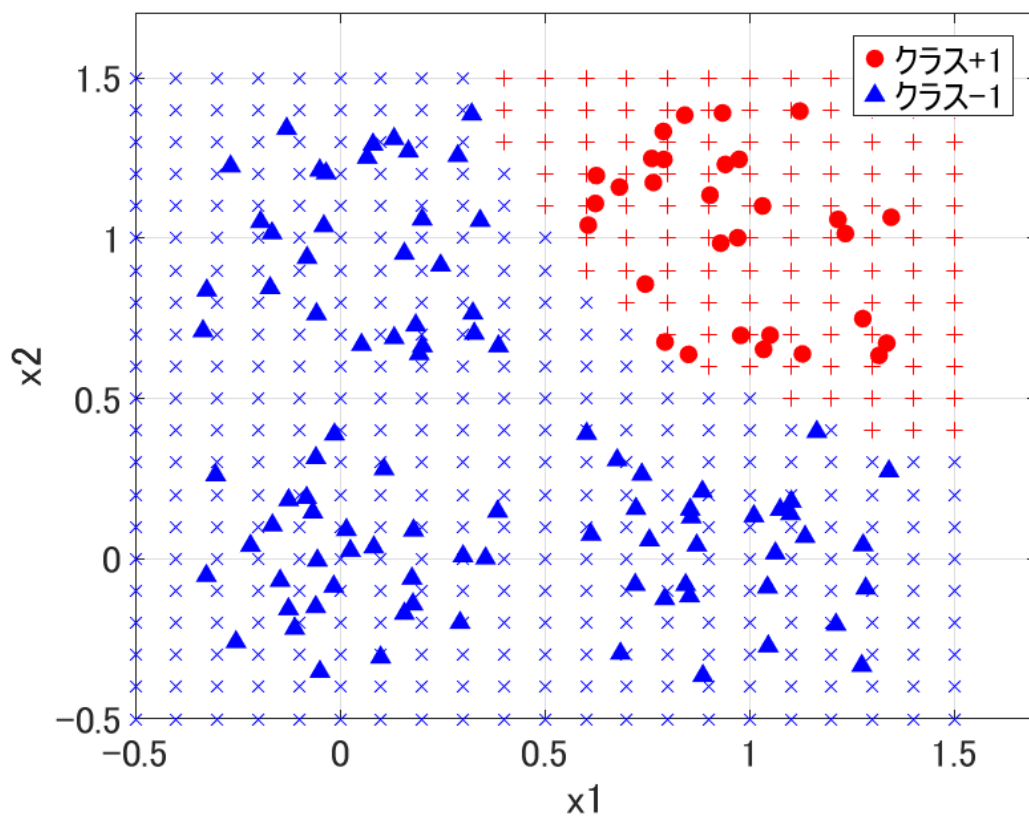


図 4.4: 3 層 NN, dimlayer=50, numEpochs=200, er=0.4

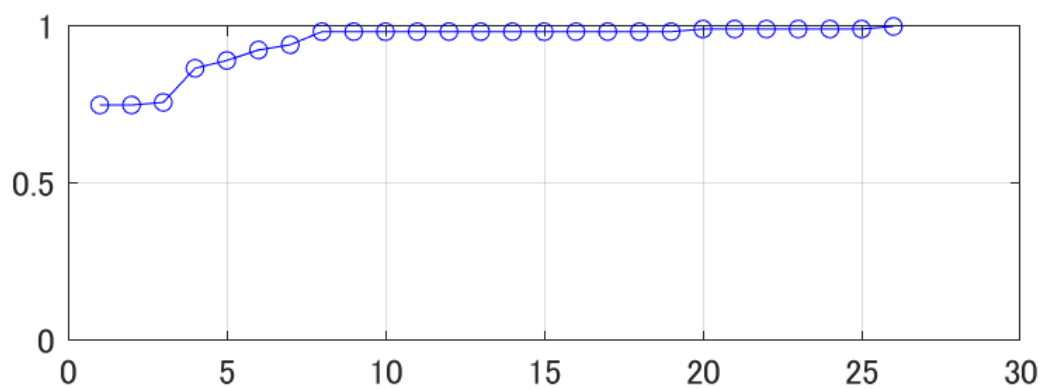


图 4.5: 3 層 NN, dimlayer=50, numEpochs=200, er=0.4

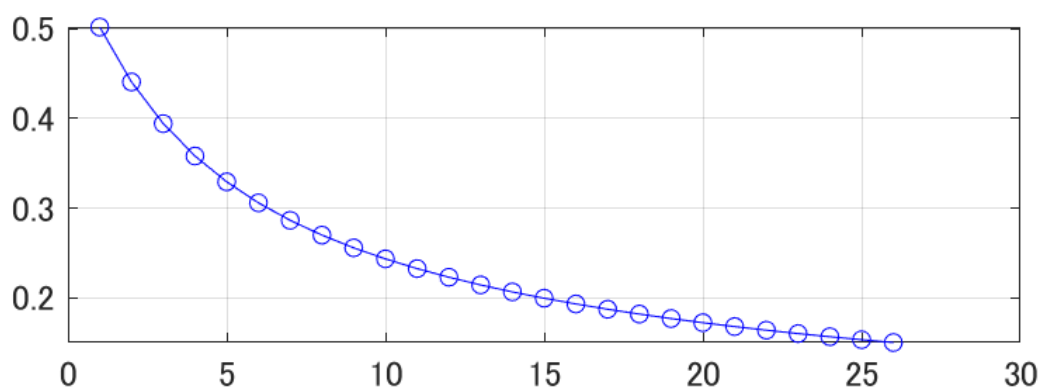


图 4.6: 3 層 NN, dimlayer=50, numEpochs=200, er=0.4

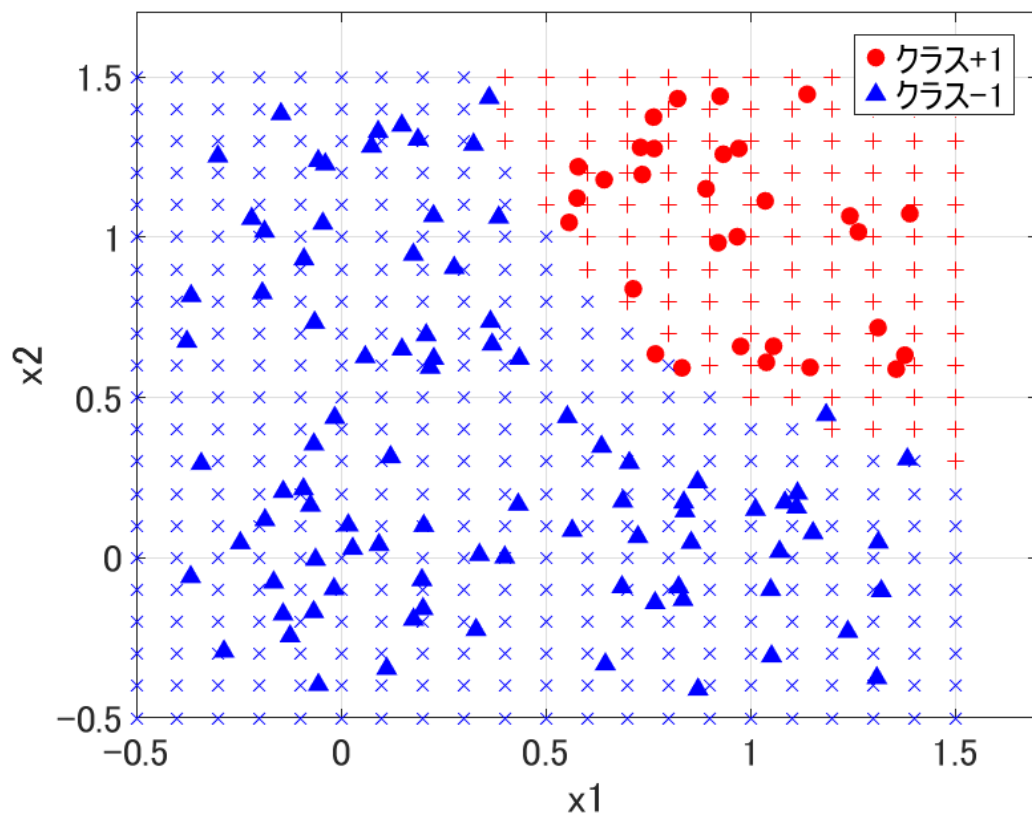


図 4.7: 3 層 NN, dimlayer=50, numEpochs=200, er=0.45

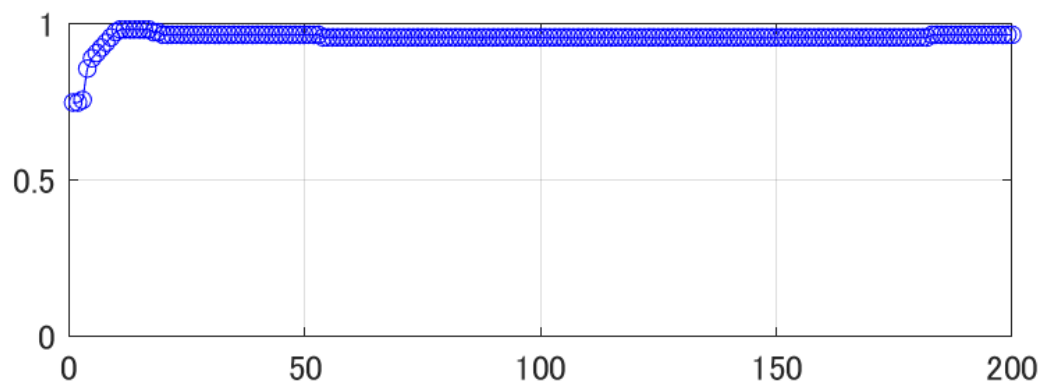


図 4.8: 3 層 NN, dimlayer=50, numEpochs=200, er=0.45

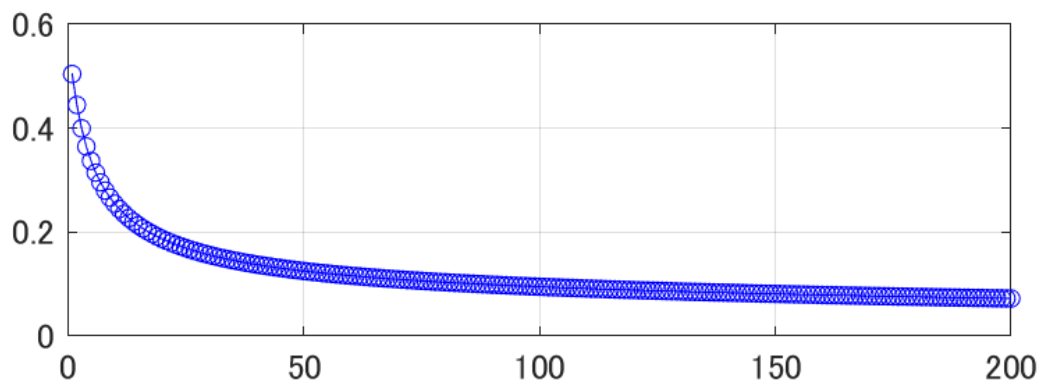


図 4.9: 3 層 NN, dimlayer=50, numEpochs=200, er=0.45

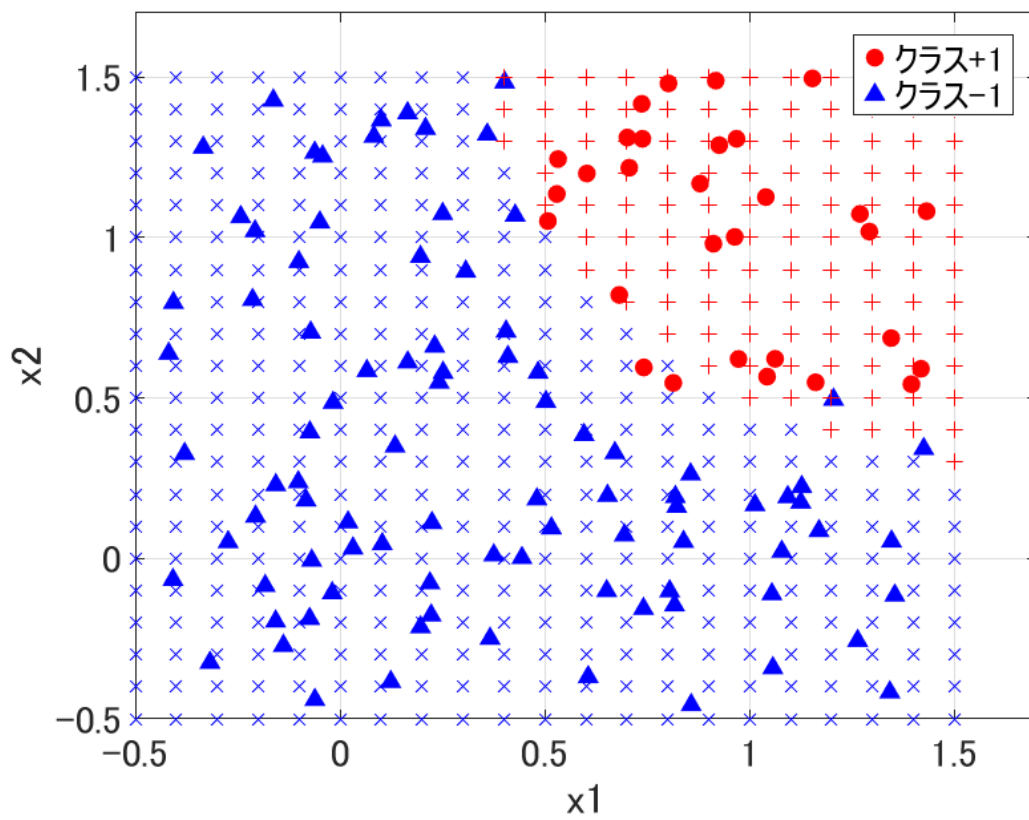


図 4.10: 3 層 NN, dimlayer=50, numEpochs=200, er=0.5

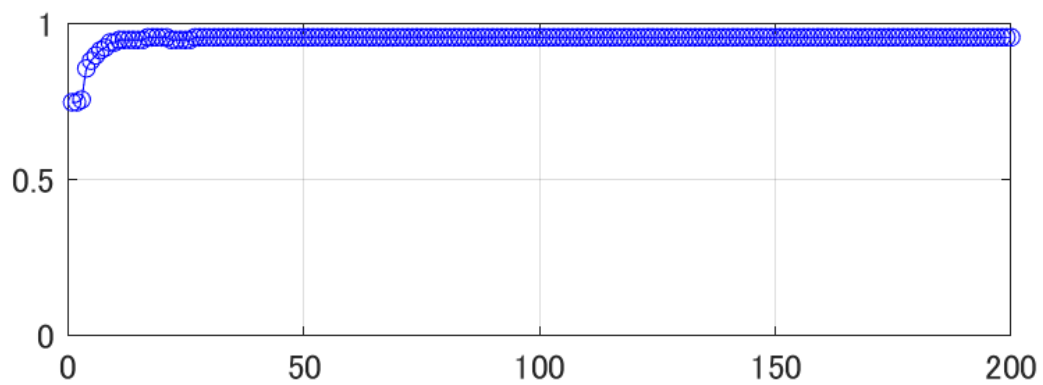


図 4.11: 3 層 NN, dimlayer=50, numEpochs=200, er=0.5

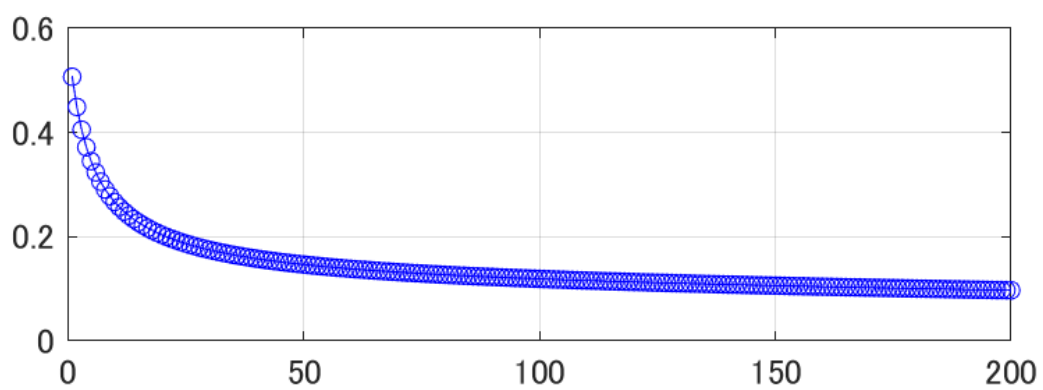


図 4.12: 3 層 NN, dimlayer=50, numEpochs=200, er=0.5



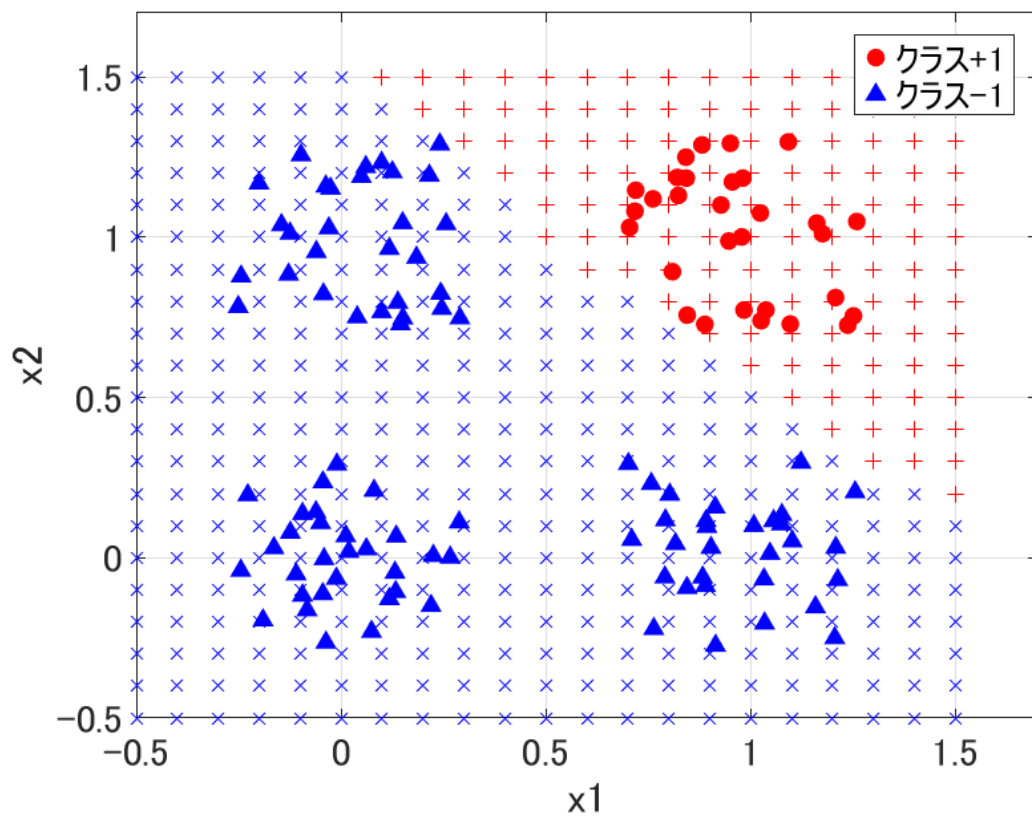


図 4.13: 3 層 NN, dimlayer=100, numEpochs=200, er=0.3

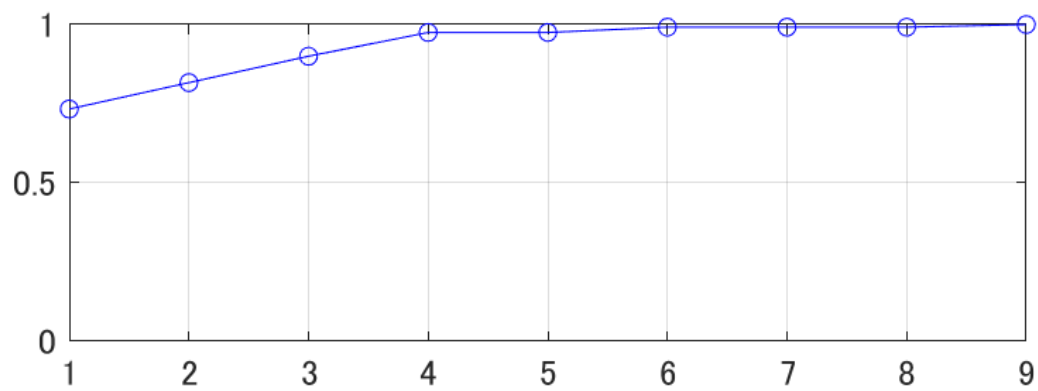


図 4.14: 3 層 NN, dimlayer=100, numEpochs=200, er=0.3

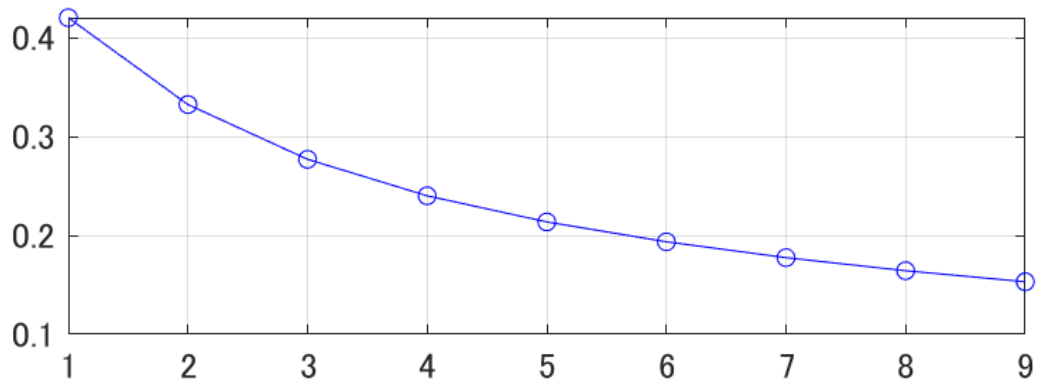


図 4.15: 3 層 NN, dimlayer=100, numEpochs=200, er=0.3

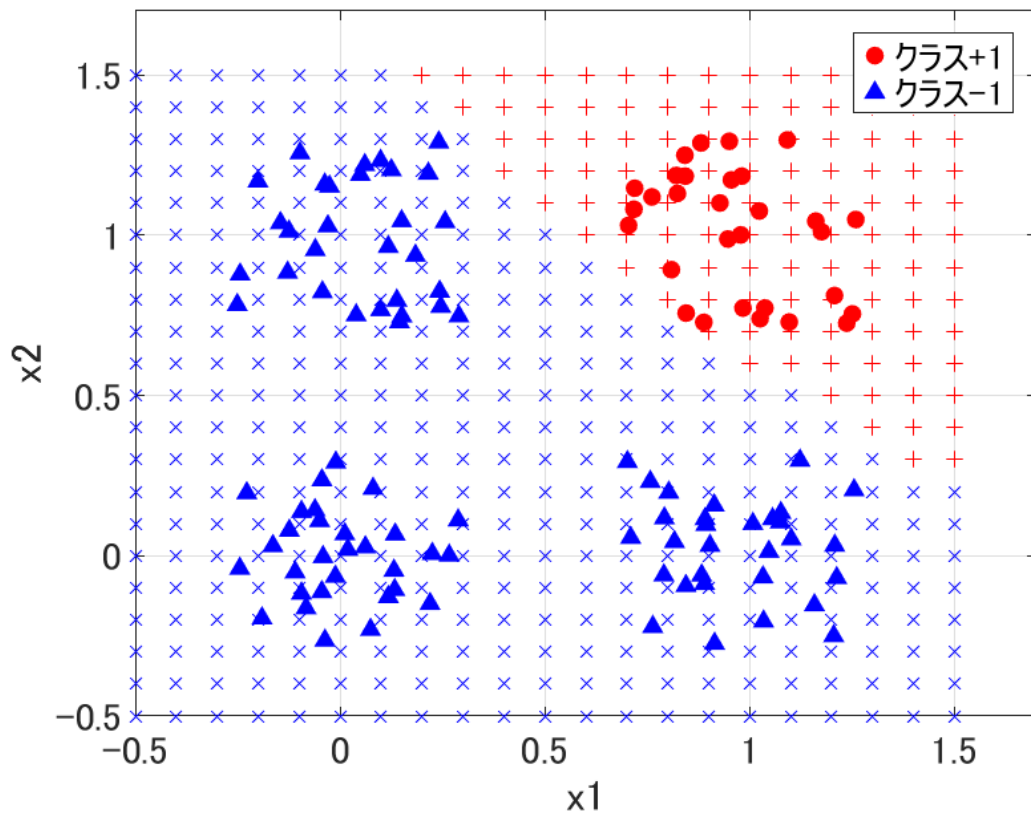


図 4.16: 3 層 NN, dimlayer=200, numEpochs=200, er=0.3

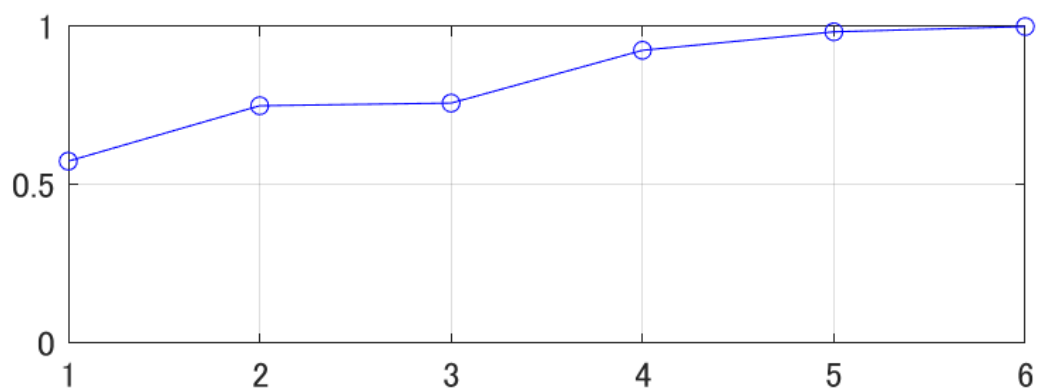


図 4.17: 3 層 NN, dimlayer=200, numEpochs=200, er=0.3

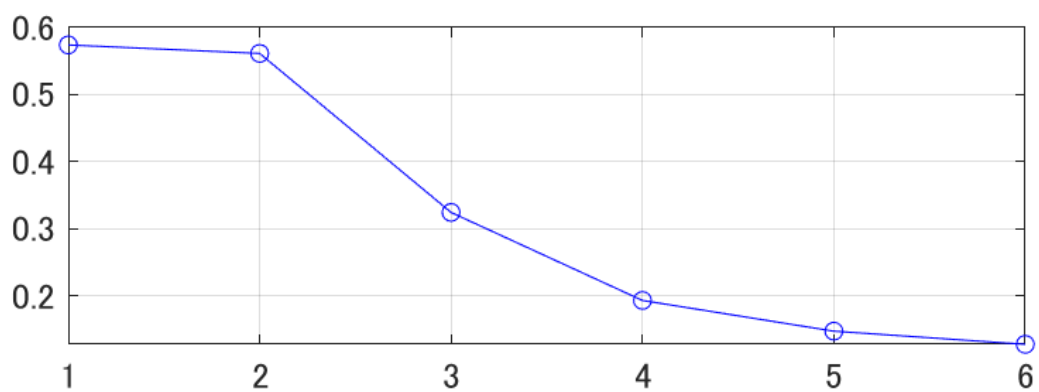


図 4.18: 3 層 NN, dimlayer=200, numEpochs=200, er=0.3

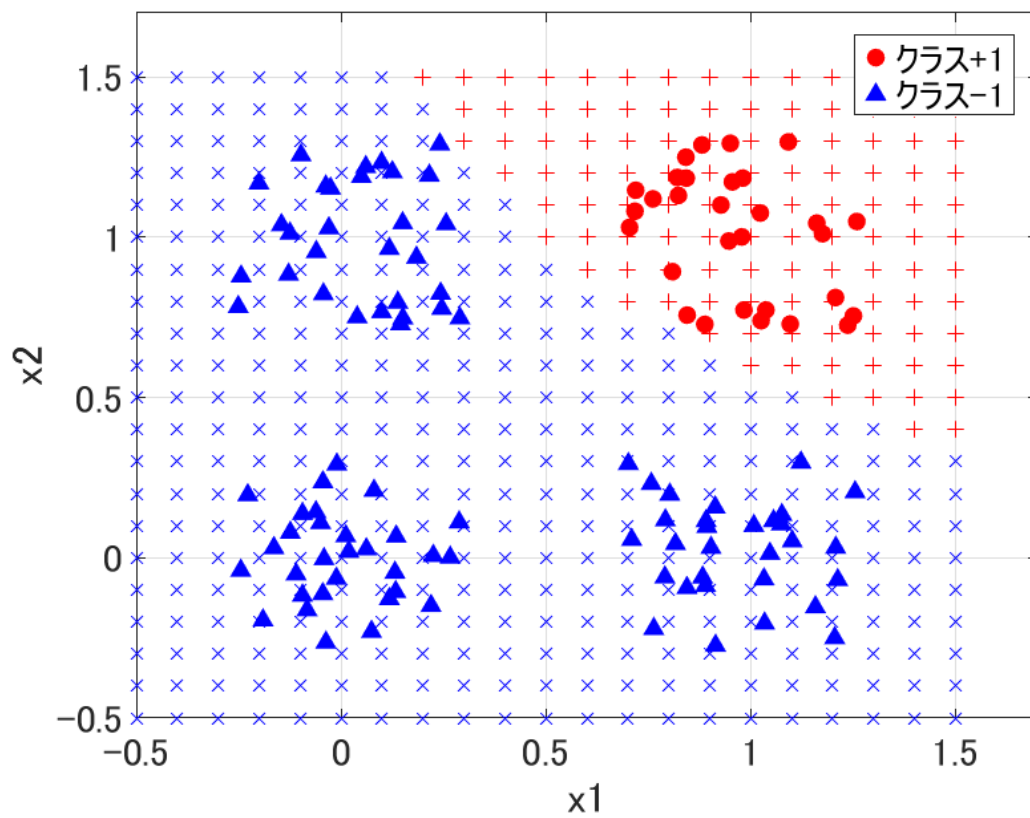


図 4.19: 3 層 NN, dimlayer=500, numEpochs=200, er=0.3

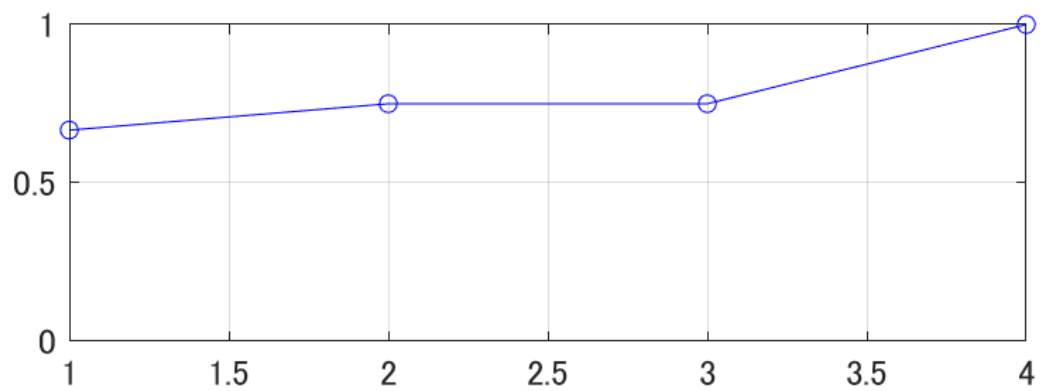


図 4.20: 3 層 NN, dimlayer=500, numEpochs=200, er=0.3

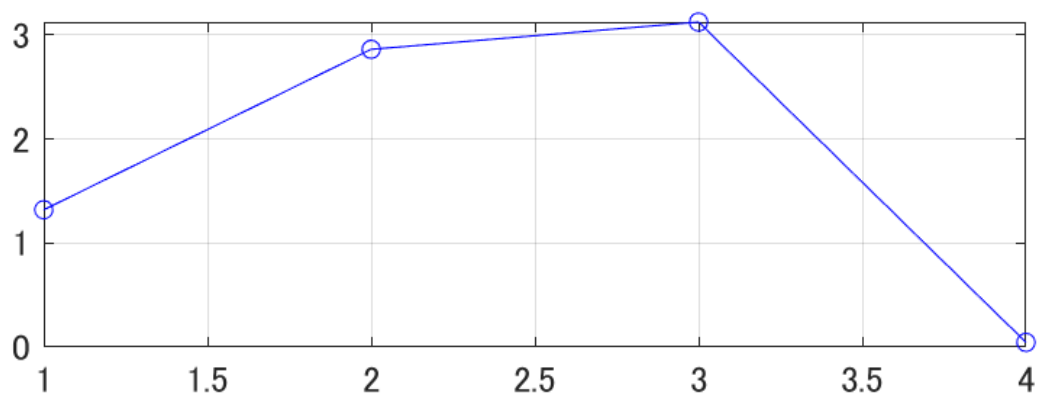


図 4.21: 3 層 NN, dimlayer=500, numEpochs=200, er=0.3

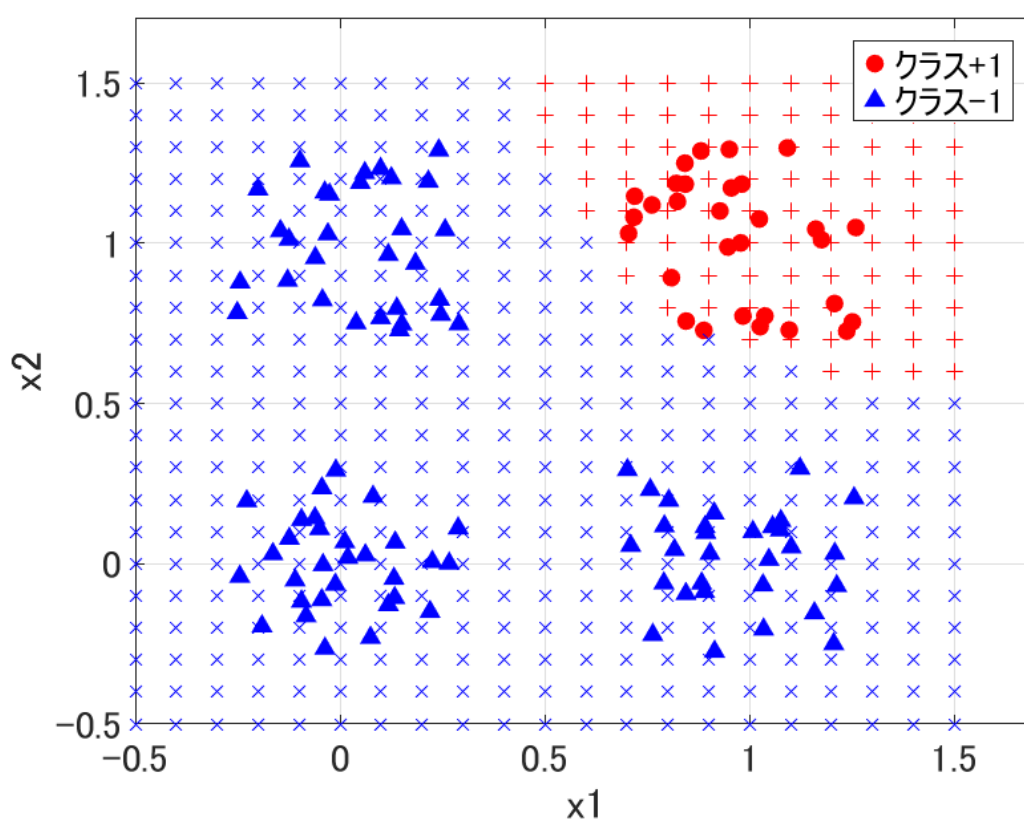


図 4.22: 3 層 NN, dimlayer=500, numEpochs=100, er=0.3

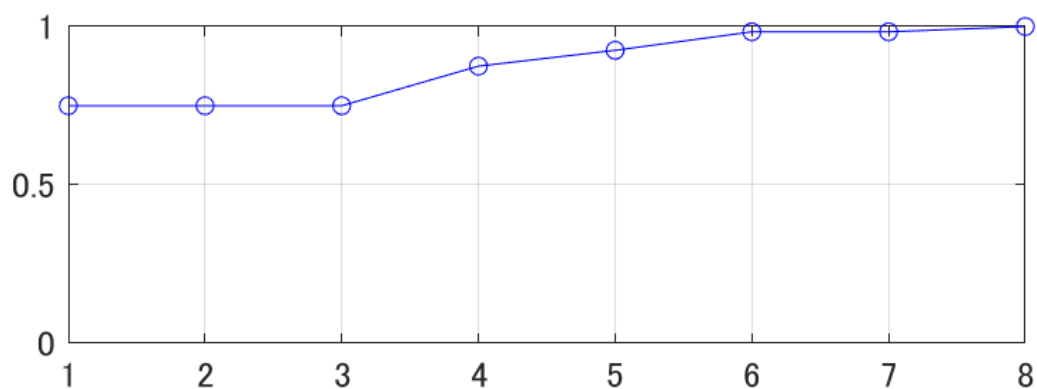


図 4.23: 3 層 NN, dimlayer=500, numEpochs=100, er=0.3

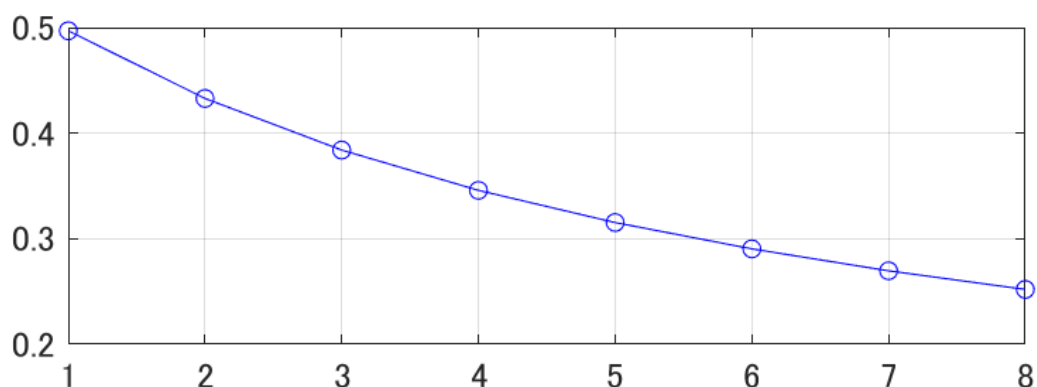


図 4.24: 3 層 NN, dimlayer=500, numEpochs=100, er=0.3

## 4.2 考察 (a)

誤差  $er$  と  $numEpochs$  が同じであるときに、中間ノードが大きくなれば正解率が 1 になるまでのエポック数が小さくなることはすべてに当てはまった。中間ノードの値が大きくなれば分類精度が高くなっていることがわかった。しかし中間ノードの値が大きくなったとしても分類精度は変わっていない。これは前者が偶発的だった可能性がある。中間ノードの変化は分類精度と正解率が 1 になるまでのエポック数両方に変化をもたらす。具体的には中間ノードが大きくなれば分類精度が高く、エポック数が小さく、中間ノードが小さくなればその逆が起こることがわかった。

## 4.3 演習課題 3.1 (b)

作成した 3 層ニューラルネットワークのプログラムを 4 層ニューラルネットワーク（2 層の中間層）に拡張せよ。そして、中間層 1、2 のノード数を 50、100、200、500 などの組み合わせで変化させ、分類精度の違いについて確認せよ。また、中間層 1 層では完全に分類できなかった場合でも、中間層を 2 層にすることにより完全に分類で

きるようになる場合があることも確認せよ。

## 5 第4週：手書き文字数字認識への拡張

Matlab の手書き数字ベースを用い、「0、1、2、3、4、5、6、7、8、9」の10個の数字に分類する10クラス分類プログラムを作成する。

### 5.1 演習課題 4.1

(a) 上記のプログラムを作成し、中間層のノード数やエポック数を変化させ、テストデータの分類精度が高くなるように工夫せよ。なお、今回の学習は各エポックの計算に時間がかかるので時間の使い方に注意せよ

### 5.2 演習課題 4.2

(a) `tryPredict` を実行し、様々な数字を入力し、その推定能力を確認する。